

Extraordinary Adventures Engine

Engine Design Document

Generated 2026-05-23 from `doc/sources/design/` (00-06)

Engine design — index

The **Extraordinary Adventures Engine** is a C++20 / SFML engine for third-person SCUMM-style point-and-click adventure games, scripted in Lua and configured with YAML. These documents are the single source of truth for the engine; the implementation follows them, not the other way around.

Reading order

Doc	Covers
01 — Engine requirements	Scope, requirements R1-R8, implementation/platform constraints, out-of-scope list.
02 — Architecture overview	Layers and dependency rule, Lua bridge (sol2), runtime spine, EngineContext, scene types, manifest, coordinate spaces + camera, persistent state / GameState, implementation guidelines.
03 — 2D game concepts	Generic 2D layer: main loop, scene contract, resources, spritesheets / animated / composite sprites, geometry + pathfinding, settings, audio.
04 — Point & click concepts	Genre layer: rooms, background layers / regions, camera, z-order, hotspots / affordances, avatars, commands + SCUMM panel, dialog, speech, inventory.
05 — Scripting API	The Lua surface: game wiring, cast, room / dialog scripts, full API reference, error handling.
06 — Data formats	Complete field reference for every YAML / Lua data file (<i>in progress</i>).

Working documents (not canonical design): [migration notes](#) and the [design review / decision record](#).

Architecture at a glance

Game	Lua scripts + assets + game manifest	no C++
Point & Click	room view, title, cutscenes, rooms, hotspots, avatars, SCUMM panel, dialog	pac::pnc
Generic 2D	spritesheets, animated sprites, composite sprites	pac::gfx
Core	window, loop, input, resources, audio, settings, Scene, SceneManager, geometry	pac::core pac::geom

Lua scripting bridge exposes a controlled API to scripts

Dependency rule: a layer may depend only on the layers below it, never above. Game scripts use only the Lua API. See [Architecture overview](#) for the full rule and folder layout.

Requirement traceability

Where each requirement from 01 is specified.

Req	Summary	Specified in
R1	Classic SCUMM experience	04 (genre scenes, commands + SCUMM panel, dialog, inventory); 05 (room scripts).
R2	Scripting-only authoring	02 (keep static data out of Lua, scripting bridge); 03 (Lua integration); 05 (whole document); 06 (data formats).
R3	Localization-ready, Spanish first	04 (hotspot id vs name, speech, localized connectors); 05 (cast names vs ids); 03 (settings language design-for).
R4	Voice-over ready	04 (speech stable line id); 05 (talk line-id note).
R5	Languages, frameworks, tools	01 R5; 02 (binding mechanism / sol2).
R6	Fixed virtual resolution + scaling	02 (coordinate systems and display pipeline); 01 R6.
R7	Operating systems	01 R7; 02 (build layout, platform handling).
R8	Save / load	01 R8; 02 (persistent state / GameState); 05 (state API).

Glossary

Coordinate spaces — *World* is the room-sized space for geometry, hotspots, objects, and avatars; *Virtual* is the fixed design resolution where UI, speech, and dialog draw; *Window* is the physical OS output. Chain: world → camera → virtual → letterbox → window. The camera maps world into the *scenery viewport* — the top region of virtual space (~85%); the SCUMM panel occupies the region beneath it, not an overlay.

Scene — a top-level application state listed in the manifest (e.g. `TitleScreen`, `StoryText`, `RoomScene`), switched via the `SceneManager`.

Room — a place inside the `RoomScene`, loaded by id from `rooms/<id>.{yaml, lua}` and switched with `change_room`. Rooms are not manifest entries.

Character — a cast entry: a stable `id`, display `name`, speech style, and an `appearance`. Lives in `cast.yaml`.

Appearance — a typed visual definition (`animated_sprite`, `composite`, or future `skeletal`) bound to a character; supplies the drawing strategy's parameters plus genre extras like `shadow`.

Avatar — the in-room visual and spatial instance of a character. The player avatar is a persistent world entity; NPC avatars are room-scoped.

AnimatedSprite / CompositeSprite — the generic-2D (`pac::gfx`) drawing strategies an appearance can use: a single animated sprite, or a hierarchy of animated sprites attached through anchors.

Anchor / pivot — a named frame-local point used for attachment; the *pivot* is the anchor whose world position the sprite's position sets.

Background layer — one image (or color) in a room's layered background, drawn at an explicit `z`. Distinct from an architecture *layer*.

Region — a changeable part of the background with named states (e.g. drawer `shut / open`), swapped with `set_region_state`.

Object — an active sprite placed in the room (visual only); interactivity is added by binding a hotspot to it.

Hotspot — an interactive area with a hit polygon or a `bind` to an object / region, an `approach` point, `affordances`, and Lua verb handlers.

Affordance — a verb the UI may offer for a hotspot or item. Affordances gate UI selection; the Lua handler decides the outcome.

Verb / Command — the player builds a *command* `verb param1 param2?`; the *command model* is language-independent and formatted for display by the UI.

Point / Zone / Walkable / Obstacle — a *point* is a named coordinate; a *zone* is a trigger polygon (room exits / scripted events); the *walkable* polygon is the room's navigable area; *obstacles* are non-walkable polygons inside it.

ZDrawable — anything drawn in the scenery; the room view stable-sorts ZDrawables by ascending `z` each frame.

State — *global state* (`set_state`, cross-room facts), *room state* (`set_room_state`, current room only), and *region state*. Values are scalars (bool / number / string) for the MVP.

GameState — the single engine-owned object that is the entire save payload (see [02](#)).

EngineContext — the struct of borrowed, app-owned service references passed to scenes (display, resources, audio, settings, scripting, scenes, log, dev).

Manifest — `game.yaml`: engine config, scene list, and entry scene.

Conventions

- All engine YAML data files carry an optional integer `version:` (default 1), bumped when a format changes incompatibly.
- Internal `ids` are ASCII, stable, and script-friendly; display `names` may contain spaces, accents, and localized text.
- Resource references are logical paths relative to `resources.src`, never platform filesystem paths.
- The Lua API uses `snake_case`; state keys use dotted names (`mummy.awake`).
- Engine-emitted UI strings (verb labels, connectors, menu labels) are looked up by key in the strings resource; game-content strings (`names`, speech, dialog) stay inline. No user-facing string is hardcoded in C++ (R3).

Engine requirements

This section defines the requirements for the first implementable version of the engine. The reader is assumed to be familiar with classic SCUMM-style adventure games. Detailed design for each subsystem is specified in the following sections.

Requirement tags. Each requirement carries a scope tag: *[MVP]* — required for the first release; *[design-for]* — not built in the MVP, but the architecture shall not preclude it; *[constraint]* — a fixed implementation or platform decision. Requirement IDs are stable references and do not imply priority or document order.

Functionality

R1 — Classic SCUMM experience. *[MVP]*

The engine shall deliver a classic third-person point-and-click adventure experience inspired by SCUMM games. The committed core features are:

- verb-based interaction;
- inventory;
- dialog with NPCs;
- navigation across multiple rooms;
- scripted room behavior and puzzle logic.

The MVP shall prioritize clarity and authoring consistency over novelty. Modern presentation features such as high-resolution backgrounds, layered scenery, parallax, and shader effects may be supported when they naturally fit the design, but they must not weaken the classic command-based interaction model.

R2 — Scripting-only authoring. *[MVP]*

A complete standard game shall be authorable without compiling game-specific C++. Game logic is written in Lua against a high-level engine API; static game data is declared in YAML. Artists and writers should be able to work from templates and fill in room files, dialog files, and object behavior without modifying the engine.

The engine shall ship template files for the expected SCUMM behavior:

- game manifest;
- cast definition;
- room YAML + Lua pair;
- dialog tree;
- global game logic.

Exception — custom interactions. Mini-games or special scenes that fall outside the classic point-and-click model may require dedicated code. The engine shall provide an extension point for these through custom scene types. Whether custom scenes are implemented in C++, Lua, or both remains a design-for decision.

R3 — Localization-ready, Spanish first. *[MVP: Spanish only · design-for: multi-language]*

The MVP ships in Spanish with no runtime language selector. The design shall not preclude adding other Western languages later.

Even in the MVP:

- no user-facing or in-game string shall be hardcoded in C++;
- fonts and text rendering shall support accented characters and common Western punctuation, including á, é, í, ó, ú, ñ, ü, and ç;
- word wrapping shall not rely on Spanish-specific assumptions;
- internal identifiers shall be separated from display names.

Engine-emitted UI text (verb labels, command connectors, built-in menu labels) is satisfied in the MVP by a single UI strings resource declared in the manifest; see [data formats](#). A language→file map and a runtime selector are the design-for path to multi-language support.

R4 — Voice-over ready. *[MVP: text-only · design-for: voice-over]*

The MVP is text-only. The design shall allow voice-over to be added later without restructuring dialogs or speech:

- the design shall not preclude assigning a stable identifier to every spoken line (the MVP may use inline text; dialog `once`-options are already keyed stably);
- an audio clip may later be attached to a spoken line;
- text display shall be synchronizable with audio playback;
- speech and audio shall be skippable as a single unit.

Voice-over is per-language and therefore depends on R3.

R8 — Save / load. *[MVP]*

The engine shall let the player save the game state and restore it later. A saved game shall capture at least:

- current top-level scene, when relevant;
- current room inside the room-view scene;
- player/avatar position and orientation;
- inventory contents;
- global scripted state;
- per-room scripted state;
- relevant room region states;
- dialog state required by `once` options or equivalent flags.

Decisions for the first implementation:

- save-file format: YAML through `yaml-cpp`, with a `save_version` integer; a compact binary format and save thumbnails are design-for;
- save slots: three manual slots plus one autosave slot;
- autosave policy: autosave on room change;
- triggering: saves are made from a menu plus engine autosave — there is no script-facing `save_game()` in the MVP (design-for);
- storage location: the per-user writable path (see [2D game concepts](#)).

The full persistent state model is specified under "Make persistent state explicit" in [the architecture overview](#).

Implementation and design constraints

R5 — Languages, frameworks, and tools. *[constraint]*

- Engine: C++20 with SFML 2.6.
- Scripting: Lua 5.4, embedded through the sol2 binding library.
- Configuration and text-based data files: YAML through `yaml-cpp`.
- Asset and authoring tools: Python.
- Authoring UI: prefer web-based tools when practical.

A web-based UI refers to offline authoring tools, not to the game runtime. The game runtime is a native SFML application.

R6 — Fixed virtual resolution with aspect-preserving scaling. *[MVP · constraint]*

Each game is authored against a fixed virtual resolution declared in the manifest under `resolution:`. "Fixed" means fixed for a given game, not an engine-wide constant. All gameplay coordinates, object positions, geometry, and input events use this virtual coordinate space.

The game may run fullscreen or windowed at any physical resolution supported by the platform. The engine maps between physical window coordinates and virtual coordinates automatically. The virtual image is scaled uniformly to preserve aspect ratio:

- if the window is wider than the virtual aspect ratio, vertical bars are shown on the left and right;
- if the window is taller or narrower than the virtual aspect ratio, horizontal bars are shown on the top and bottom.

Platform

R7 — Operating systems. *[constraint]*

- Development environment: Ubuntu 24.04 LTS.
- Primary release target: Windows 10/11 x64.
- Linux is a possible later release target.

Out of scope for the first version

The following are out of scope for the first implementation:

- character switching, as in *Maniac Mansion*, *Day of the Tentacle*, or *Thimbleweed Park*;
- runtime multi-language UI, beyond being design-ready for it;
- voice-over playback, beyond being design-ready for it;
- 3D rendering;
- networking or multiplayer;
- a full visual IDE for authoring;
- runtime modding or user-generated content support.

Lightweight in-game edit-mode helpers and standalone Python tools are allowed, but they are not the same as a full authoring IDE.

A general view of the architecture

The engine is C++20 + SFML, scripted in Lua, and configured with YAML. Exact versions and platform constraints are defined in [Engine requirements](#). This section specifies the C++ architecture, the dependency rules, the runtime spine, the manifest, and the display pipeline.

Layers and dependency rule

The engine is split into layers with a single one-directional dependency rule: a layer may depend only on the layers below it, never above.

Game	Lua scripts + assets + game manifest	no C++
Point & Click	room view, title, cutscenes, rooms, hotspots, avatars, SCUMM panel, dialog	pac::pnc
Generic 2D	spritesheets, animated sprites, composite sprites	pac::gfx
Core	window, loop, input, resources, audio, settings, Scene, SceneManager, geometry	pac::core pac::geom

Lua scripting bridge exposes a controlled API to scripts

Layer	Namespace	Owns	Depends on
Core	pac::core , pac::geom	Window, main loop, input, resources, audio services, settings, diagnostics, geometry, Scene , SceneManager	SFML only
Generic 2D	pac::gfx	Spritesheets, animated sprites, composite sprites	Core
Point & Click	pac::pnc	Room view, title, cutscene scene, rooms, hotspots, avatars, SCUMM panel, dialog runtime	Generic 2D, Core
Game	—	Manifest, YAML data, Lua scripts, assets	Lua API only

The dependency direction is enforced by folder structure and namespaces:

```
include/engine/core/  
include/engine/geom/  
include/engine/gfx/  
include/engine/pnc/
```

Separate static libraries are not required for the first implementation. The folder and namespace structure is enough to signal and enforce the intended architecture.

Core

Core is the platform harness. A game developer does not edit Core to make a standard point-and-click game.

Core owns:

- window and resolution management;
- the fixed-timestep main loop;
- physical-to-virtual input mapping;
- resource management;
- audio services;
- settings service;
- the diagnostics service (logging and assertions);
- basic graphics primitives;
- the geometry module;
- the `Scene` interface;
- the `SceneManager` and scene stack.

Generic 2D

Generic 2D contains reusable building blocks for 2D games. These concepts are not specific to point-and-click adventures.

Generic 2D owns:

- `Spritesheet`;
- `AnimatedSprite`;
- `CompositeSprite`;
- sprite anchors and sequence playback.

The scene interface and scene manager remain in Core. Concrete scenes such as the room view, title screen, and cutscene player belong to the Point & Click layer because they encode genre behavior.

Point & Click layer

The Point & Click layer owns the genre-specific model:

- room-view scene;
- title scene;
- cutscene scene;
- room model;
- room YAML loader;
- room Lua binding;
- hotspots and affordances;

- avatars;
- SCUMM panel;
- command builder;
- dialog runtime;
- speech presentation.

See [Third-person point & click concepts](#).

Lua scripting bridge

The Lua scripting bridge lets game scripts drive the engine through a restricted API. Scripts do not access C++ objects directly except through registered wrapper objects and functions.

The bridge has two directions:

Direction	Purpose	Examples
Engine → script	Invoke lifecycle hooks and verb handlers.	<code>on_load</code> , <code>on_unload</code> , <code>on_zone_enter</code> , <code>hotspots.drawer.open</code>
Script → engine	Drive engine services through the Lua API.	<code>talk</code> , <code>change_room</code> , <code>play_music</code> , <code>avatar(id):move_to</code>

The Lua API uses `snake_case`. The full API is specified in [The scripting API](#).

Binding mechanism

The bridge is implemented with [sol2](#), a header-only binding layer over Lua 5.4. The conventions below are part of the design, not just an implementation detail:

- The engine embeds one `lua_State`, owned by the scripting service and wrapped in a `sol::state`. Nothing else creates Lua states.
- Engine → script calls go through `sol::protected_function`. A failed call is reported through the diagnostics service and never aborts the frame; in development builds it fails loudly (see the error handling rules in [The scripting API](#)).
- Script → engine functions are registered as flat `snake_case` globals, matching the API reference in 05.
- Handles such as `avatar(id)` return a lightweight `sol::usertype` that stores **only a stable id**, never a raw C++ pointer. The C++ side resolves the id to an object on each call, so a handle that outlives a room change fails loudly instead of dangling.
- C++ holds no Lua references across frames except the coroutine threads owned by the script scheduler.

Script task ownership

Lua coroutines are transient runtime work owned by the script scheduler. Every scheduled task belongs to one explicit script scope:

Scope	Lifetime
Scene	One top-level scene instance.
Room	The currently loaded room inside <code>RoomScene</code> .
Dialog	One active dialog.
Global	The current <code>RoomScene</code> / game session.

`spawn(fn)` inherits the current script scope. Room hooks and hotspot handlers spawn room-scoped tasks; dialog callbacks spawn dialog-scoped tasks; cutscene scripts spawn scene-scoped tasks. Cross-room global tasks are design-for and must be explicit if added later.

When a scope ends, the scheduler cancels all tasks owned by that scope before the engine destroys objects from that scope. Cancellation removes the task from the scheduler and it is never resumed. Lua cleanup code is not run as part of task cancellation in the MVP; cleanup belongs in normal lifecycle hooks such as `on_unload` and dialog `on_exit`.

Lifecycle boundaries:

- `change_room` calls the outgoing room's `on_unload`, cancels the outgoing room scope, destroys room-scoped objects, then loads the next room.
- Room script reload in development mode is treated as `on_unload` + room-scope cancellation + script reload + `on_load`.
- Dialog end calls `on_exit`, cancels the dialog scope, then returns the room view to `Command`.
- Scene replacement or pop cancels the scene scope and its child scopes before destroying the scene.
- Load-game cancels the current `RoomScene` script scopes, deserializes `GameState`, enters the restored scene / room, and runs the normal `on_load`. Saved games never resume old coroutines.

Runtime spine

Startup and scene transitions are data-driven. A game declares its scenes in a YAML manifest. The engine constructs scenes from that manifest using a factory keyed by scene type.

Startup sequence

1. Load the game manifest.
2. Create the window and the letterboxed virtual view.
3. Initialize global services: resources, audio, settings, and scripting.
4. Register built-in scene types.
5. Register custom scene types, if the game provides any.
6. Construct and enter the manifest's `entry` scene.
7. Run the fixed-timestep main loop.

Scenes vs. rooms

Scenes and rooms are different levels of the runtime model.

Concept	Meaning	Switched by	Declared in
Scene	Top-level application state.	<code>SceneManager::goto_scene</code> , <code>push_scene</code> , <code>pop_scene</code>	Manifest
Room	Place inside the point-and-click room-view scene.	<code>change_room("id")</code>	Room files

Rooms are not manifest entries. The manifest contains one `RoomScene`, and that scene loads rooms from the game's room directory.

Engine context and services

Global services are created once at startup (step 3 of the startup sequence) and owned by the application for the entire process lifetime. Scenes never own global services; they borrow them through an `EngineContext` passed by reference at construction.

```
struct EngineContext {
    Display&      display;      // virtual<->window mapping + view
    ResourceCache& resources;    // logical-path asset access
    AudioServices& audio;       // MediaPlayer + SoundPlayer
    Settings&     settings;     // persisted player settings
    Scripting&    scripting;     // sol::state + coroutine scheduler
    SceneManager& scenes;       // goto / push / pop
    Diagnostics&  log;          // logging + asserts
    const DevFlags& dev;        // development flags from the manifest
};
```

Ownership and lifetime:

- Services are owned by the application object, constructed before the first scene and destroyed after the last, so the references are always valid.
- A scene may store the `EngineContext` or copy out only the service references it needs. A scene shall not extend service lifetime, cache raw pointers to services, or share services with background threads.
- Point & Click components (`DialogRuntime`, `CommandBuilder`, `RoomLoader`, ...) receive only the specific services they use, not the whole context. This keeps the dependency graph explicit and the headless modules narrow.

Input is not a service in the context: input reaches the focused scene through `handle_event`, and physical-to-virtual coordinate mapping is done by `Display` before dispatch. A `Clock` / time service may be added later if needed.

Scene types, outcomes, and wiring

A scene type defines named outcomes. The manifest wires each outcome to another scene id or to a reserved token such as `QUIT`.

A scene decides when an outcome happens; the manifest decides where that outcome leads. This keeps scene implementations reusable.

A scene is constructed with:

- its `YAML` parameters ;
- the `EngineContext` described in "Engine context and services" above.

Built-in scene types:

Type	Purpose
<code>TitleScreen</code>	Main menu/title screen.
<code>StoryText</code>	Text-based cutscene.
<code>RoomScene</code>	SCUMM-style point-and-click gameplay.

Custom scene types are the extension point for mini-games and special interactions outside the standard point-and-click model.

Game manifest

The game manifest is the single source of game-level configuration.

```
# my_game.yaml
version: 1
resolution:
  width: 1280
  height: 720

window:
  fullscreen: false
  width: 1280
  height: 720

resources:
  src: games/themummy/data

strings: strings/es.yaml

settings:
  audio:
    music_volume: 0.8
    sfx_volume: 0.8

development:
  edit_mode: false
  show_walkboxes: false
  show_hotspots: false
  allow_room_reload: true

entry: title

scenes:
  - id: title
    type: TitleScreen
    parameters:
      new_game: intro_cutscene
      exit: QUIT

  - id: intro_cutscene
    type: StoryText
    parameters:
      script: scripts/intro.lua
      on_finish: room_view

  - id: room_view
    type: RoomScene
    parameters:
      cast: cast.yaml
      logic: game.lua
      inventory: inventory.yaml
      inventory_logic: inventory.lua
      rooms: rooms
      start_room: study
      player: julia
      font: fonts/DepartureMono-Regular.otf
```

Manifest fields

Field	Scope	Meaning
<code>version</code>	Optional	Data-format version integer (default 1). See data formats .
<code>resolution</code>	Required	Virtual design resolution.
<code>window</code>	Required	Initial physical window mode and size.
<code>resources.src</code>	Required	Resource source: filesystem directory for MVP; archive file design-for.
<code>strings</code>	Required	UI strings resource for engine-emitted text (verb labels, connectors, menu labels). One file in the MVP; a language→file map is design-for. See data formats .
<code>settings</code>	Optional	Default player-facing settings. User settings may override these.
<code>development</code>	Optional	Development-only flags. Not persisted as player settings.
<code>entry</code>	Required	Initial scene id.
<code>scenes</code>	Required	Scene list and outcome wiring.

`resources` shall describe where assets are loaded from. It shall not contain specific assets such as UI fonts. Fonts are requested by the scene, UI, or script that needs them through logical resource paths.

`development` shall contain development-only flags. These flags are distinct from player-facing settings, which are persisted by the settings service.

`strings` points at the UI strings resource. Every user-facing string the engine itself generates — verb labels, command connectors, and built-in menu labels — is looked up by key in this resource, never hardcoded in C++ (R3). Strings authored as game content — hotspot and item names, speech, and dialog lines — remain inline in their own files and are not part of this resource.

Resource source

`resources.src` identifies the root source for logical resources.

src kind	Scope	Meaning
Directory	MVP	Filesystem resource backend. Useful during development.
Archive file	Design-for / post-MVP	Packed resource backend. Useful for distribution.

All game assets are referenced by logical paths relative to `resources.src`, for example:


```
backgrounds/study.png
fonts/DepartureMono-Regular.otf
rooms/study.yaml
anim/hero_body.anim.yaml
```

The resource layer resolves logical paths to bytes. Game code and scripts shall not construct platform-specific filesystem paths.

Coordinate systems and display pipeline

The engine uses three coordinate spaces.

Space	Meaning
World	Room-sized coordinate system for room geometry, points, hotspots, objects, and avatars. A room may be larger than the virtual resolution.
Virtual	Fixed design coordinate system declared by the game manifest. UI such as the SCUMM panel, speech, and dialog options live here.
Window	Actual physical OS window or fullscreen output. This may vary per machine.

The transform chain is:

```
world —camera→ virtual —letterbox scale→ window
```

The camera is a world-space rectangle the size of the **scenery viewport** — the room-view layout's top region (~85% of the virtual height; the SCUMM panel occupies the remaining ~15%), not the full virtual resolution. Each frame the room view updates the camera, clamps it to the room bounds, and maps the world into the scenery viewport; the SCUMM panel and other UI then draw in virtual space in their own regions beneath it, not as an overlay on top of the scenery. The engine then scales the whole virtual image uniformly into the physical window while preserving aspect ratio, filling unused space with bars.

Pointer input is mapped from physical pixels back into virtual coordinates first. A pointer inside the scenery viewport is mapped on into world coordinates through the inverse camera transform for room and hotspot hit testing; a pointer inside a UI region such as the SCUMM panel is handled in virtual coordinates. The mapped input is then dispatched to the focused scene.

Example: a 1280×720 virtual game on a 2560×1440 window scales exactly 2×. On a 3440×1440 ultrawide monitor, it scales 2× to fit the height, occupies 2560 physical pixels horizontally, and leaves pillarbox bars on the sides.

Scene stack

The `SceneManager` owns a stack of scenes. The top scene is focused.

Operation	Meaning
<code>goto_scene(id)</code>	Replace the active scene and collapse overlays.
<code>push_scene(id)</code>	Push an overlay scene above the current one.
<code>pop_scene()</code>	Remove the top scene and return to the one below.

By default, only the top scene receives input and updates. Scenes draw from bottom to top. A pushed overlay may be opaque or transparent.

Implementation guidelines

This section defines implementation rules for the first engine refactor/rewrite. The goal is not to over-engineer the engine, but to prevent the prototype from growing again through accidental architecture.

The design document is the source of truth. Existing prototype code may be reused when it matches the target design, but it shall not constrain the design.

Build vertical slices first

Implementation should proceed through playable vertical slices, not isolated subsystems.

A useful milestone is a minimal game with:

- one manifest-driven startup flow;
- one title scene;
- one room scene;
- two or three rooms loaded from YAML + Lua;
- a controllable player avatar;
- hotspots;
- the SCUMM command panel;
- inventory;
- one dialog;
- one simple puzzle spanning at least two rooms.

A subsystem is considered useful only when it helps make that slice playable.

Separate engine, games, tests, and experiments

The repository should make the boundary between the engine and each executable explicit. A standard game should eventually be authored without compiling new C++ code, but the build layout shall still allow games, tests, experiments, and custom scenes to have their own small C++ entry points when needed.

Recommended project layout:

```

.
├── lib/
│   ├── include/engine/      # public engine headers
│   ├── src/                 # engine implementation
│   └── CMakeLists.txt       # engine library target
├── games/
│   ├── themummy/
│   │   ├── CMakeLists.txt   # example game executable
│   │   ├── main.cpp         # minimal entry point
│   │   └── data/            # manifest, Lua, YAML, assets
│   └── CMakeLists.txt
├── experiments/
│   ├── ...                  # throwaway or exploratory executables
│   └── CMakeLists.txt
├── tests/
│   ├── ...                  # unit/integration tests
│   └── CMakeLists.txt
└── CMakeLists.txt           # top-level build

```

The engine target owns the reusable implementation. Games, tests, and experiments link against the engine target. During the first implementation phase, compiling executables directly against the engine sources is acceptable if it keeps the build simple, but this is a build-system convenience, not a design dependency.

Each game, test, or experiment may provide:

- a minimal `main.cpp` that initializes the engine and selects a manifest;
- its own game data: Lua scripts, YAML files, assets, dialogs, and room files;
- optional custom `Scene` subclasses when the interaction falls outside the standard point-and-click model;
- its own CMake target.

The top-level `CMakeLists.txt` should include one CMake file per group (`lib`, `games`, `tests`, `experiments`) and each group should define its own sub-targets. This keeps the common engine build separate from the executables that exercise it.

Example target structure:

```

pac_engine      # engine library
pac_themummy    # example game executable
pac_room_tests  # test executable
pac_dialog_tests # test executable
pac_experiment_x # exploratory executable

```

The build shall support Ubuntu and Windows builds on the host platform being used. Cross-compilation is not required. Platform-specific differences should be contained in CMake options, dependency setup, or small platform adapters, not in game scripts.

Tests should be first-class build targets, written with the doctest framework (single-header, registered with CTest). When supported by the compiler and platform, the test configuration should enable sanitizers such as AddressSanitizer and UndefinedBehaviorSanitizer to catch leaks, memory errors, and undefined behavior early. Sanitizers are development tools and are not part of release game builds.

This layout supports the long-term scripting-only authoring goal without making it artificially strict during engine development. A game that needs only standard point-and-click behavior should not need custom C++ code; a game that needs a custom mini-game or experiment may still add a small executable or custom scene in a controlled place.

Dependencies and acquisition

Dependency	Kind	Linux	Windows
SFML 2.6	compiled	<code>apt libsFML-dev</code> <code>(find_package(SFML 2.6))</code>	<code>vcpkg sfml</code>
Lua 5.4	compiled	<code>apt liblua5.4-dev</code>	<code>vcpkg lua</code>
yaml-cpp	compiled	<code>apt libyaml-cpp-dev</code> <code>(find_package(yaml-cpp))</code>	<code>vcpkg yaml-cpp</code>
sol2	header-only	<code>CMake FetchContent (pinned)</code>	<code>CMake FetchContent (pinned)</code>
doctest (tests)	header-only	<code>CMake FetchContent (pinned,</code> <code>with vendored fallback)</code>	<code>CMake FetchContent (pinned,</code> <code>with vendored fallback)</code>
micropather (design-for, unused)	vendored if a grid A* is adopted	<code>third_party/micropather/</code>	<code>third_party/micropather/</code>

Acquisition rules:

- Compiled libraries come from the system package manager — `apt` on Linux, `vcpkg` on Windows. `CMake` picks them up through `find_package`, so a single `CMake` invocation works on both platforms once the packages are present.
- Header-only libraries (`sol2`, `doctest`) are pulled with `CMake FetchContent` at a pinned version on both platforms, so there is no per-platform divergence and no dependency on an `apt` package or `vcpkg` port.
- `doctest` may additionally be **vendored** as a single-header fallback under `tests/_vendor/`. The build prefers the vendored copy when present; this is useful in offline / air-gapped environments where `git` is unreachable.
- `micropather` is a design-for alternative, vendored under `third_party/` only if a coarse-grid A* is ever adopted; it ships as bare source files without a `CMake` target. The MVP ships a built-in visibility-graph A* behind the `find_path` interface, so `micropather` is not currently vendored (see [pathfinding](#)).

Keep `RoomScene` as an orchestrator

`RoomScene` coordinates the room-view state, but it shall not become the owner of all point-and-click logic.

It should delegate to smaller components:

Component	Responsibility
RoomRuntime	Loaded room data, current room state, zones, points, hotspots, avatars.
RoomLoader	Load <room>.yaml and <room>.lua , bind static data to script handlers.
RoomRenderer	Draw background layers, regions, objects, avatars, speech, and debug overlays.
CommandBuilder	Maintain the SCUMM command-building state machine.
CommandDispatcher	Convert a completed command into movement + Lua handler invocation.
ScummPanel	Render verbs, inventory pages, and command bar; translate UI clicks into intents.
InventoryModel	Store inventory items and expose inventory operations.
DialogRuntime	Run dialog trees and manage dialog-state transitions.
SpeechManager	Display spoken lines and later connect them to voice-over.
AvatarController	Move avatars, face targets, play actions, and report movement completion.

RoomScene may own these components and coordinate them, but it should not absorb their logic.

Keep the command system independent from the UI

The SCUMM panel is one way to build commands. It is not the command system.

The command system shall accept a command model such as:

```
struct Command {
    Verb verb;
    ObjectRef param1;
    std::optional<ObjectRef> param2;
};
```

This allows tests, debug tools, scripted commands, and future input methods to reuse the same dispatch path.

Keep static data out of Lua

Lua scripts define behavior. They should not define room geometry, visual layout, asset paths, or hotspot polygons.

Static authoring data belongs in YAML:

- manifest;
- cast;
- room layout;
- geometry;
- hotspots;
- affordances;
- background layers;
- regions;
- object placement;

- avatar start positions.

Lua may refer to these entities by id and decide what happens when the player interacts with them.

Make persistent state explicit

Persistent game state shall live only in explicit engine-managed stores:

- current scene and current room;
- player/avatar position and orientation;
- inventory contents;
- global world state;
- per-room state;
- declared object or region state.

Lua local variables, closures, and currently running coroutines are transient and shall not be part of save/load. Scripts must write every persistent fact through the explicit state, inventory, region, or dialog-flag stores.

Scripts that need persistent information must use `get_state` / `set_state` or `get_room_state` / `set_room_state`.

All of this lives in one engine-owned `GameState`, independent of which scene or room is currently loaded — room instances are transient views over it. `GameState` is the entire save payload:

```
GameState {
  current_scene_id  : string
  room_view: {
    current_room_id : string
    player: { x, y, facing, appearance_id }
  }
  inventory          : list<item_id>
  global_state       : map<string, Value>          # set_state
  room_state         : map<room_id, map<string, Value>> # all rooms, not only loaded
  region_states      : map<room_id, map<region_id, state_id>>
  dialog_flags       : map<dialog_id, consumed once-options>
  save_version       : int
}
```

- `room_state[room_id]` persists whether or not the room is loaded; `set_room_state` writes to the current room's entry, created lazily and never cleared on unload.
- `region_states` persists per room. A region's YAML `initial` state applies on a new game or the first time a room is entered; on load-game the persisted `region_states` win over `initial`. The `on_load` hook that runs after restore observes the restored states, not `initial`.
- `Value` is a scalar (boolean, number, string); see the scripting API state rules.
- Consumed `once` dialog options are keyed by `(node_id, option_index)` for the MVP; a stable explicit option id is design-for.
- Saving serializes `GameState` to YAML in the per-user location. Loading deserializes it, enters `current_scene_id`, and — for `RoomScene` — loads `current_room_id` and reseats the player at

the saved position.

- In the MVP, saving (the menu save and the room-change autosave) happens only while `RoomScene` is active; `current_scene_id` is still recorded so saving from other scenes is a design-for extension rather than a format change.
- On restore, `on_load` runs again to rebuild transient room setup from persisted state; there is no separate `on_restore` hook.

Save format, slots, and autosave policy are specified in [R8](#).

Prefer simple MVP behavior over speculative generality

When a feature has both an MVP version and a design-for version, implement the MVP version first.

Area	MVP	Design-for
Resources	Filesystem backend + cache	Packed archive.
Backgrounds	Layers and z-order	Parallax and shaders.
Sprites	Animated sprites with anchors	Advanced composite sprites.
Movement	Visibility-graph pathfinding around obstacles	Navmesh/funnel smoothing, dynamic obstacles.
Dialogs	Text choices	Voice-over synchronization.
Settings	Minimal display/audio support	Complete settings UI.
Save/load	Explicit state snapshot, three manual slots, one autosave slot	Thumbnails.

Do not implement a design-for feature unless it is needed by the current vertical slice or it prevents a known architectural dead end.

Use debug tools as part of the MVP

Debug tooling is part of productivity, not polish.

The engine should support development flags for:

- showing walkable areas;
- showing obstacles;
- showing hotspots;
- showing approach points;
- showing avatar anchors;
- showing z-order values;
- showing current command-builder state;
- showing room/world state;
- reloading the current room script;
- jumping to a room;
- adding/removing inventory items.

These flags belong in development configuration, not in player-facing settings.

Keep layer boundaries strict

The dependency rule is part of the design.

- Core shall not know about point-and-click concepts.
- Generic 2D shall not know about verbs, inventory, rooms, or dialogs.
- Point & Click may use Generic 2D and Core.
- Game scripts may use only the Lua API.
- Standard games shall not require C++ changes.

If a feature seems to require violating a layer boundary, the design should be reviewed before implementation continues.

Keep logic separable from rendering

Game logic shall not depend on a live graphics context. The following modules must be constructable and exercisable in a headless test with no window, so they can be unit-tested and reused outside the renderer:

- `pac::geom` — points, polygons, point-in-polygon, pathfinding;
- the command model and command-builder state machine;
- the dialog runtime — tree walking, `when / run / once / silent, END`;
- the state stores — global, per-room, region, and dialog flags;
- the inventory model;
- the YAML loaders — manifest, room, cast, anim, dialog — producing validated data structures.

Rendering (`ZDrawable::draw`, SCUMM panel layout, speech presentation) may require a graphics context and is exercised through the example game and edit mode rather than unit tests. This separation is the practical meaning of the layer rule: point-and-click logic must not reach for `sf::RenderTarget`. The list above is a starting contract and may grow or shrink as the engine evolves.

Treat examples as test candidates

Every example in the design document should be implementable.

When possible, examples should become regression tests or sample content in the example game. The command-builder transition table and the dialog execution steps in [Third-person point & click concepts](#) are natural table-driven tests. If an example cannot be implemented cleanly, either the design or the example is wrong.

Avoid hidden behavior

Implicit behavior should be minimized.

Acceptable implicit behavior:

- default fallback responses for missing verb handlers;
- default standing animation after an action;
- default room entry point when no specific entry point is provided.

Risky implicit behavior:

- automatically inventing object interactions;
- silently accepting invalid commands;
- saving state that was not explicitly declared;
- resolving missing resources without visible diagnostics;
- allowing Lua scripts to depend on physical filesystem paths.

In development builds, authoring errors should fail loudly with clear diagnostics.

Migration notes

Prototype divergences and refactoring tasks are tracked separately in [migration notes](#). They are not part of the target design.

2D game concepts

This section defines concepts common to several types of 2D games. They make up the Generic 2D layer and parts of Core. Point-and-click-specific behavior is specified in [Third-person point & click concepts](#).

Initialization, main loop, and scenes

The engine harness — startup, main loop, window, aspect scaling, and scene transitions — is fixed engine code. Authors do not modify it to create a standard game. A scene is the C++ extension point for behavior outside the standard engine model, such as a custom mini-game.

Main loop

The engine uses a fixed-timestep loop. Each iteration performs:

1. poll input events;
2. map physical coordinates to virtual coordinates;
3. dispatch events to the focused scene;
4. update the focused scene using a fixed `dt`;
5. draw the scene stack.

The fixed timestep is 60 Hz ($dt = 1/60$ s). The loop accumulates elapsed wall-clock time and runs zero or more fixed `update(dt)` steps per rendered frame, so gameplay stays deterministic and refresh-rate independent: a 144 Hz display draws more often but still advances the simulation at 60 Hz, and a slow frame catches up with several steps (capped to avoid a spiral of death).

Scene contract

Every scene implements:

Method	Called	Purpose
<code>enter()</code>	Once, when becoming active or pushed	Acquire resources, initialize state, start music.
<code>leave()</code>	Once, when replaced or popped	Release scene-owned resources, stop scene-owned effects.
<code>handle_event(event)</code>	Per input event, focused scene only	Turn input into scene intent.
<code>update(dt)</code>	Fixed timestep, focused scene only	Advance scene state.
<code>draw(target, viewport)</code>	Each frame	Render in virtual coordinates.

A scene is constructed with:

- its manifest `parameters`;

- the `EngineContext` of shared services (display/view, resources, audio, settings, scripting, scene manager, diagnostics, dev flags), specified in [the architecture overview](#).

Resources

The resource layer provides one way to access every game asset:

- textures;
- fonts;
- music;
- sound effects;
- shaders;
- Lua scripts;
- YAML files;
- atlas metadata;
- dialog and room data.

Assets are named by logical paths relative to the game's resource source. A logical path is not a filesystem path.

Examples:

```
backgrounds/study.png
rooms/study.yaml
scripts/intro.lua
fonts/DepartureMono-Regular.otf
```

Resource source interface

A resource source opens a readable byte stream for a logical path.

Backend	Scope	Meaning
Filesystem source	MVP	Reads loose files under the resource root.
Packed archive source	Design-for / post-MVP	Reads assets from a single indexed archive.

Game code, engine code above the resource layer, and scripts shall use logical paths. They shall not concatenate platform-specific filesystem paths.

Caching

Fully-loaded assets are cached by logical path. Repeated requests for the same asset return the same loaded instance.

Asset kind	Loading behavior
Texture	Fully loaded and cached.
Font	Fully loaded and cached.
Sound buffer	Fully loaded and cached.
Shader	Fully loaded and cached.
YAML/Lua data	Loaded through the resource layer; caching depends on subsystem needs.
Music	Streamed as one active track; not cached as a full buffer.

Packed archive design-for

The packed backend is not required for the MVP, but the resource interface shall allow it. A packed archive may be implemented as either:

- a custom file with an index mapping logical path to offset, size, and flags;
- a standard archive format such as zip.

The exact format is an open question. The MVP shall not assume loose filesystem files outside the resource layer.

Spritesheets, animated sprites, and composite sprites

SFML provides textured sprites. The engine builds three concepts on top:

```
Spritesheet → AnimatedSprite → CompositeSprite
```

Spritesheet

A spritesheet is an atlas texture plus YAML metadata describing named frames.

Each frame has:

Field	Meaning
<code>id</code>	Stable frame identifier.
<code>rect</code>	Rectangle inside the atlas.
<code>source_rect</code>	Optional original source rectangle before packing. For traceability only.
<code>anchors</code>	Named frame-local points used for pivots and attachments.

Example:

```
image: body.png
size: { width: 1024, height: 1024 }

sprites:
  - id: frame_000
    rect: { x: 109, y: 2, width: 107, height: 236 }
    source_rect: { x: 121, y: 38, width: 107, height: 236 }
    anchors:
      walking_pivot: { x: 54, y: 236 }
      head_mount: { x: 53, y: 42 }
```

Anchors are frame-local pixels with origin at the frame's top-left corner.

AnimatedSprite

An `AnimatedSprite` plays named sequences of frames from a spritesheet. It supports the usual sprite rendering operations:

- translation;
- rotation;
- scale;
- opacity;
- blend mode;
- shader;
- sequence selection;
- looping and non-looping playback.

Positioning by anchor

An animated sprite designates one anchor as its pivot. Setting the sprite's position places the current frame's pivot at that world position. This keeps feet, hands, or other anchors stable across frames without per-frame offset logic.

Example animation file:

```
# body.anim.yaml
version: 1
spritesheet: body.yaml
pivot: walking_pivot

sequences:
  idle_down:
    loop: true
    frames:
      - { sprite: frame_000, duration: 0.12 }

  walk_down:
    loop: true
    frames:
      - { sprite: frame_001, duration: 0.10 }
      - { sprite: frame_002, duration: 0.10 }
      - { sprite: frame_003, duration: 0.10 }
      - { sprite: frame_004, duration: 0.10 }

  pick_up_down:
    loop: false
    frames:
      - { sprite: frame_010, duration: 0.08 }
      - { sprite: frame_011, duration: 0.08 }
```

Sequence completion

A non-looping sequence shall support both:

Form	Purpose
Callback	C++ code can be notified when playback finishes.
Blocking script call	Lua coroutine can wait until playback finishes.

Example Lua usage:

```
avatar("player"):play_until_end("pick_up_down")
```

The call yields the coroutine and resumes it when the sequence ends.

CompositeSprite

A `CompositeSprite` is a hierarchy of animated sprites attached through anchors. It is the default model for composed characters.

Example: a body sprite exposes a `head_mount` anchor. A head sprite uses its `neck_pivot` anchor. Each frame, the head's `neck_pivot` is aligned to the body's `head_mount`.

```
# hero.composite.yaml
root:
  id: body
  animation: anims/hero_body.anim.yaml

children:
  - id: head
    parent: body
    animation: anims/hero_head.anim.yaml
    parent_anchor: head_mount
    child_anchor: neck_pivot
```

Children may animate independently of parents. For example, the body can walk while the head talks or turns.

Composite sprites shall expose the same high-level rendering operations as animated sprites where meaningful: position, scale, opacity, and draw.

Lua integration

The engine embeds a single Lua state and runs game logic as cooperative coroutines. Scripts can write straight-line logic using blocking-looking calls without freezing the main loop.

Example:

```
spawn(function()
  talk("hero", "I should check the drawer.")
  avatar("hero"):move_to("at_drawer")
  avatar("hero"):play_until_end("open_down")
  wait(0.3)
  talk("hero", "Empty. Of course.")
end)
```

Each blocking call yields to the script scheduler. The main loop continues to update and draw.

Script task states

Wait kind	Resumes when	Script call
Ready	Next scheduler update	Newly spawned task or explicit yield.
Timer	N seconds elapse	<code>wait(seconds)</code>
Event	Named event is emitted	<code>wait_event(name)</code> / <code>emit(name)</code>
AvatarMove	Avatar finishes moving	<code>avatar(id):move_to(target)</code>
Animation	Animation sequence finishes	<code>avatar(id):play_until_end(sequence)</code>
Speech	Speech line or text page finishes or is skipped	<code>talk(speaker, text_or_line_id)</code> / <code>show_text(text)</code>
Done	Never	Task finished.

Script task scopes and cancellation

Each task belongs to one script scope. `spawn(fn)` inherits the scope of the hook, handler, dialog callback, or scene script that calls it.

Spawn site	Default scope	Cancelled when
Room lifecycle hook or hotspot handler	Current room	<code>change_room</code> , room reload, <code>RoomScene</code> leave, load restore.
Dialog callback / option <code>run</code>	Current dialog	Dialog ends, is cancelled, or <code>RoomScene</code> leaves.
<code>StoryText</code> / cutscene script	Current scene	Cutscene finishes, is skipped, or scene is replaced.
Engine-owned global startup work	Current game session	<code>RoomScene</code> leaves, new game starts, save is loaded, or process exits.

Cancellation is immediate from the scheduler's point of view: a cancelled task is removed and is never resumed. The engine detaches any waiter associated with the task:

Wait kind	Cancellation behavior
Timer / Event	Remove the wait; scoped events from later rooms do not resume old tasks.
Speech	Hide or skip the active speech line.
Animation	Stop waiting; the animation falls back according to avatar defaults.
AvatarMove	Stop movement if the avatar belongs to the cancelled scope; player re-seat rules still apply during room changes.
Camera tween	Stop the tween and restore the room camera policy.
Room transition	Once begun, finish the engine transition, but do not resume a cancelled caller.

The MVP does not run Lua finalizers when cancelling a task. Room cleanup belongs in `on_unload`; dialog cleanup belongs in `on_exit`. Development diagnostics may log cancelled task ids and their wait kinds.

C++ to Lua

The engine invokes Lua hooks for:

- scene events;
- room lifecycle;
- zone enter/exit;
- hotspot verb handlers;
- dialog callbacks.

Lua to C++

Lua scripts use the registered API specified in [The scripting API](#). Scripts shall not rely on C++ internal classes or memory ownership.

Geometry

The geometry module (`pac::geom`) is a small foundational module with no game or genre dependencies.

Types

Type	C++	YAML / Lua representation
Point	<code>pac::geom::Point</code> / <code>sf::Vector2f</code>	<code>{ x: 10, y: 20 }</code>
Polygon	<code>std::vector<Point></code>	Ordered list of points

The module shall support serialization to and from YAML and Lua tables.

Operations

Operation	Purpose
<code>distance(a, b)</code>	Proximity checks.
<code>distance_squared(a, b)</code>	Threshold tests without square root.
<code>point_in_polygon(p, poly)</code>	Hit testing, zones, walkable checks.
<code>segment_intersection(a, b, c, d)</code>	Navigation and tool support.
<code>polygon_bounds(poly)</code>	Debug rendering and spatial indexing.

Walkable area and obstacles

A room's navigable space is inside its walkable polygon and outside all obstacle polygons.

```
is_walkable(p) = point_in_polygon(p, walkable)
                && !point_in_any_polygon(p, obstacles)
```

A room has one walkable polygon and zero or more obstacle polygons.

Pathfinding

Pathfinding returns an ordered list of waypoints from a start point to a destination, staying inside the walkable polygon and outside all obstacles.

The engine wraps the pathfinder behind a stable interface:

```
find_path(start, destination) -> list<Point>
```

The MVP ships a built-in **visibility-graph** A* pathfinder, computed per query with no room-load preprocessing:

- **Nodes** are the start, the destination, and one node inset a few pixels off each walkable concave corner and each obstacle corner, on the free-space side. The inset keeps graph edges clear of the boundary instead of grazing it (where the inside/outside test is

ambiguous) and is what lets a path round both obstacles and concave walkable pockets (e.g. an L-shaped room).

- **Edges** connect any two nodes whose connecting chord stays in free space: its midpoint is walkable and outside every obstacle, and it crosses neither the walkable boundary nor an obstacle edge. Edge cost is euclidean distance.
- **Search** is A* with the euclidean distance to the destination as an (admissible, consistent) heuristic, so the result is a shortest path through the corner graph.

Contract details that callers rely on:

- The returned list excludes the start and ends at the destination; a clear line of sight short-circuits to a single waypoint.
- A start or destination outside the walkable area is clamped to the nearest reachable point, so a click just past the boundary still routes.
- The result is **never empty**: when no corner route exists it falls back to a straight walk truncated at the first boundary/obstacle, yielding one reachable waypoint. An empty or degenerate walkable polygon is treated as ungated and yields `{destination}`.
- Obstacles are assumed **convex** — the midpoint chord test is exact for them; author a concave obstacle as a union of convex polygons.

The avatar walks the returned waypoints in order, turning at each corner.

A coarse-grid A* (e.g. the **micropather** library, vendorable under `third_party/` since it is not packaged for apt or vcpkg), a funnel / navmesh smoother, and dynamic obstacles are design-for alternatives behind the same `find_path` seam. None are required by the MVP, and micropather is not currently vendored.

Settings

Player-facing settings are preferences chosen by the player and persisted across runs. They are distinct from the game manifest and distinct from development flags.

Stored settings

Setting	Scope	Notes
Display mode	MVP	Fullscreen/windowed and physical window size.
Music volume	MVP	Applied to the music player.
SFX volume	MVP	Applied to the sound player.
Language	Design-for	MVP ships Spanish only.
Text speed	Nice-to-have	Useful for subtitle pacing.

Defaults and precedence

The manifest provides defaults. User settings override manifest defaults. On a first run with no user settings file, manifest defaults are used.

Settings are saved in a per-user writable location, not in the resource root. Examples:

- `%APPDATA%` on Windows;
- `$XDG_CONFIG_HOME` or `~/.config` on Linux.

Save files use the same general per-user storage policy.

Settings scene

The settings UI is a scene pushed over the current scene. It edits the settings service and then pops back to the previous scene. This makes the same settings available from the title screen or from an in-game pause menu.

It offers a windowed-resolution selector, a fullscreen toggle, and music volume. A display change does not happen in the scene: it is **requested** through the `Display` service (`request_mode`), and the main loop recreates the OS window between frames and reports the applied size/mode back to `Display`. This keeps the window owned by the core harness while a genre-layer scene drives the change through `EngineContext`. Because the **virtual resolution never changes** — only the letterbox does — switching modes is transparent to the game (R6): gameplay coordinates, geometry, and input mapping are unaffected.

The windowed sizes offered are aspect-matching multiples of the virtual resolution that fit the desktop, so windowed mode never shows bars. Fullscreen targets the **optimal mode for the game**: the video mode that matches the virtual resolution when the monitor supports it, otherwise the smallest mode large enough to contain it (letterboxed), falling back to the desktop mode.

Music and sound

The engine exposes two global audio services.

Service	Purpose
<code>MusicPlayer</code>	One streamed background track at a time.
<code>SoundPlayer</code>	Short overlapping sound effects.

Music player

The music player supports:

- `play(path, loop);`
- `stop();`
- replacing the current track by playing a new one;
- volume controlled by settings.

Lua API:

```
play_music("music/study.ogg", true)
stop_music()
```

Sound player

The sound player supports short overlapping sounds, such as clicks, footsteps, and object interaction sounds. Sound buffers are loaded through the resource layer and cached.

Lua API:

```
play_sound("sfx/click.wav")
stop_sounds()
```

Packed resource note

`sf::Music` streams from a file. In packed mode, the selected music track may be extracted to a temporary file and streamed from there. Other assets should load from streams directly when possible.

Migration notes

Prototype divergences and refactoring tasks are tracked separately in [migration notes](#). They are not part of the target design.

Third-person point & click concepts

This layer (`pac::pnc`) contains the components specific to third-person point-and-click adventure games. It builds on the generic 2D concepts specified in [2D game concepts](#).

Genre scenes

Scene	Purpose
<code>RoomScene</code>	Main SCUMM-style gameplay: rooms, scenery, characters, verb panel, inventory, dialog.
<code>TitleScreen</code>	Main menu/title screen customized for point-and-click games.
<code>Cutscene / StoryText</code>	Scripted text or visual sequences between gameplay scenes.

`RoomScene` is specified in the rest of this document. The two supporting scenes are summarized here.

TitleScreen

The entry menu. Parameters:

Parameter	Meaning
<code>background</code>	Optional background image (logical path).
<code>music</code>	Optional music track played while the menu is shown.
<code>new_game</code>	Outcome scene id started by "New game".
<code>exit</code>	Outcome for "Quit" (usually the reserved <code>QUIT</code> token).

Named outcomes: `new_game`, `continue` (resume the most recent save, offered only when a save exists), `settings` (push the settings scene), and `exit`. The manifest wires `new_game` and `exit`; the engine handles `continue` and `settings`.

StoryText / Cutscene

A text or scripted sequence between gameplay scenes. Parameters:

Parameter	Meaning
<code>script</code>	Lua script (logical path) that drives the sequence as a coroutine.
<code>on_finish</code>	Outcome scene id entered when the script completes.

The script may show text pages, play music, and wait, using the same coroutine API as room scripts. Text pages are shown with `show_text(text, dur?)`, a speaker-less page drawn centered in virtual space and skippable as a single unit (see [the scripting API](#)); `talk` is reserved for in-room, near-speaker speech. The scene is skippable; skipping cancels the running script

(remaining pages and waits do not run) and fires `on_finish`. Any persistent state a cutscene must guarantee should be written before its first yield or in the receiving scene, since a skipped script does not run past the skip point. Named outcome: `on_finish`.

The room

A room is a unit of place in the game world. It contains scenery, interactive contents, navigation geometry, and characters.

A room is authored as two files linked by id:

File	Purpose
<code>rooms/<id>.yaml</code>	Static definition: background, layers, geometry, points, zones, regions, objects, hotspots, affordances, initial avatars.
<code>rooms/<id>.lua</code>	Behavior: lifecycle hooks, zone hooks, hotspot verb handlers, scripted events.

The YAML file defines what exists. The Lua file defines what happens.

Example room YAML

```
version: 1
id: study
# No explicit size: the room's world bounds are derived from the background
# layers (the union of their rects, floored to the room view). See below.

background:
  color: { r: 0, g: 0, b: 0, a: 255 }
  layers:
    - id: sky
      image: backgrounds/study/sky.png
      z: 0
      interactive: false

    - id: room_back
      image: backgrounds/study/back.png
      z: 10
      interactive: false

    - id: table_front
      image: backgrounds/study/table_front.png
      z: 640
      origin: { x: 980, y: 300 } # foreground occluder, placed at native size
      interactive: false

perspective:
  top: { y: 380, scale: 0.70 }
  bottom: { y: 700, scale: 1.15 }

walkable:
  - { x: 40, y: 600 }
  - { x: 1560, y: 600 }
  - { x: 1560, y: 700 }
  - { x: 40, y: 700 }

obstacles:
  - [ {x: 700, y: 610}, {x: 900, y: 610}, {x: 900, y: 690}, {x: 700, y: 690} ]

points:
  player_start: { x: 120, y: 650 }
  at_drawer: { x: 520, y: 660 }

zones:
  - id: to_hall
    polygon: [ ... ]

regions:
  drawer:
    area: [ ... ]
    z: 520
    states:
      shut: backgrounds/regions/drawer_shut.png
      open: backgrounds/regions/drawer_open.png
    initial: shut

objects:
  lamp:
    sprite: anims/lamp.anim.yaml
    position: { x: 980, y: 520 }
    z: auto
```

```
hotspots:
  drawer:
    name: "el cajón"
    area: [ ... ]
    approach: at_drawer
    bind: region:drawer
    affordances: [ look_at, open, close ]

  lamp:
    name: "la lámpara"
    bind: object:lamp
    approach: { x: 960, y: 650 }
    affordances: [ look_at, use, pick_up ]

avatars:
  - { id: julia, start: player_start, orientation: down, player: true }
  - { id: schneider, start: at_drawer, orientation: left }
```


Example room Lua

```
local room = {}

function room.on_load()
    play_music("music/study.ogg", true)
end

function room.on_unload()
    stop_music()
end

room.on_zone_enter = function(zone)
    if zone == "to_hall" then
        change_room("hall")
    end
end

room.hotspots = {
    drawer = {
        look_at = function()
            return "Es un cajón viejo."
        end,

        open = function()
            set_region_state("drawer", "open")
            set_room_state("drawer.open", true)
            return "Lo abro."
        end,

        close = function()
            set_region_state("drawer", "shut")
            set_room_state("drawer.open", false)
            return "Lo cierro."
        end,
    },

    lamp = {
        look_at = function()
            return "Una lámpara polvorienta."
        end,

        use = function()
            return "No enciende."
        end,
    },
}

return room
```

Room contents

Content	Source	Notes
Background layers	YAML	Static or animated scenery layers.
Background regions	YAML + Lua	Changeable parts of the background, such as drawers or doors.
Objects	YAML	Active sprites placed in the room.
Characters / avatars	YAML + cast	Player and NPCs. Behavior comes from scripts.
Hotspots	YAML + Lua	Interactive areas and verb handlers.
Points	YAML	Named coordinates for starts, approach points, camera targets.
Walkable area	YAML	Main navigation polygon.
Obstacles	YAML	Non-walkable polygons inside the walkable area.
Zones	YAML + Lua	Trigger polygons for room exits or scripted events.

Background layers

A room background is a list of layers, not a single image. This supports layered composition, animation, foreground occlusion, and future shader effects.

Layer field	Meaning
<code>id</code>	Stable layer id.
<code>image</code>	Logical resource path.
<code>z</code>	Draw depth. Larger values are nearer the camera.
<code>origin</code>	Optional <code>{x, y}</code> room-space top-left where the layer is drawn (default the world origin <code>(0,0)</code>), so layers may differ in size and be placed freely — a foreground occluder, a parallax-ready backdrop, a decal.
<code>scale</code>	Optional uniform render scale about <code>origin</code> (default <code>1.0</code> = native pixel size). Aspect ratio is always preserved — layers are never distorted, only uniformly enlarged/shrunk. Mainly a development aid for sizing furniture-style occluder layers; in production layers should ship at their correct native size (<code>scale: 1</code>).
<code>interactive</code>	Whether this layer can receive pointer interactions. Usually false.
<code>visible</code>	Optional initial visibility (default <code>true</code>). Toggled at runtime with <code>set_layer_visible(id, bool)</code> ; requires the layer to carry an <code>id</code> . World bounds are derived from all layers, hidden or not, so toggling never reshapes the room.
<code>shader</code>	Optional shader resource or shader config. Design-for.
<code>animation</code>	Optional animation description. Design-for.

Use cases:

Use case	Example
Static background	Laboratory back wall.
Non-interactive animated layer	Moving clouds.
Foreground occluder	Table front drawn in front of characters.
Shader effect	Water distortion.
Interactive visual state	Prefer a <code>region</code> or <code>object</code> , not a raw background layer.

A layer may use a solid `background.color` behind all images. This is useful for transparent layers, exported foregrounds, and authoring workflows.

Layer visibility vs. regions. Toggling a whole layer with `set_layer_visible` is the right tool for show/hide of a piece of scenery that occupies its own image and depth — e.g. a foreground cart drawn with perspective, removed once it is taken away. Use a **region** instead when the *same* footprint swaps between named state images (drawer `shut / open`); use an **object** for an active sprite the player interacts with via a bound hotspot. Layer visibility is persisted per room (like region and object state), so a layer hidden by script stays hidden across room changes and save/load.

World bounds

A room has no authored `size` . Its world bounds are **derived** from the layers: each layer occupies `[origin, origin + native image size × scale)` , the world is the union of those rects anchored at `(0,0)` , and it is floored to the room-view size so the world is never smaller than the visible scenery viewport. Only the right/bottom extents grow the world; a layer at a negative `origin` spills off the top-left and is simply never scrolled to. Anything not covered by a layer shows `background.color` — leaving a gap is an authoring mistake, not a format error. This keeps the world coordinate system a consequence of the art rather than a number to keep in sync with it. Because the derivation needs image pixel dimensions, it happens render-side when textures load (`compute_room_bounds`); the headless room loader and geometry/camera logic never depend on it (the `Camera` takes explicit bounds).

Room view screen layout



The scenery continues to draw in every room-view state. The bottom panel changes according to the room-view state.

Room view states

State	Panel	Input	Use case
Command	SCUMM panel visible and enabled	Player can build commands.	Default gameplay.
Dialog	Dialog options panel	Player selects dialog options.	NPC conversations.
Blocked	Black or hidden panel	Player interaction disabled.	Scripted moments, cutscene-like actions.

The room-view state is distinct from the command-builder state machine. The command builder exists only while the room view is in `Command` state.

Z-order and `ZDrawable`

Everything that draws in the scenery participates in depth ordering through a common interface:

```
interface ZDrawable {
    float z() const;
    void draw(target) const;
}
```

Each frame, the room view gathers all `ZDrawable`s, stable-sorts them by ascending `z`, and draws them in order. Larger `z` values are nearer the camera and are drawn later.

Default `z` values:

Entity	Default <code>z</code>
Background base	Behind all sorted drawables.
Background layer	Explicit layer <code>z</code> .
Region	Explicit <code>z</code> , or the <code>z</code> of the layer named by <code>over</code> .
Object	<code>z: auto</code> (default): world-Y of the sprite pivot; numeric <code>z</code> overrides.
Avatar	Its walking pivot y coordinate.
Speech	Drawn above scenery, not sorted as world geometry.
SCUMM panel	UI layer, not part of room z-order.

An object's `z: auto` and a region's `over: <layer_id>` let scenery be authored without hand-tuning depths: the object sorts by its pivot's world-Y like an avatar, and the region sorts with the background layer it changes.

Perspective scaling

A room may define how vertical position affects avatar scale.

```
perspective:
  top:    { y: 380, scale: 0.70 }
  bottom: { y: 700, scale: 1.15 }
```

Each anchor is a floor line: a world-space `y` and the avatar render `scale` at that line. The engine linearly interpolates an avatar's scale from its walking pivot `y` between `top` and `bottom`, and clamps to the nearest anchor's scale outside the band. This applies to every avatar in the room (player and NPCs), and because an avatar scales about its walking pivot (feet), it stays planted as the scale changes. Characters are usually smaller near the top of the screen and larger near the bottom, because the bottom represents the foreground in a front-facing room.

If `perspective` is omitted, each avatar keeps its base scale (the value it was created with, `1.0` by default).

Camera

The room camera is a world-space rectangle the size of the scenery viewport — the room-view layout's top region, not the full virtual resolution (see [coordinate systems](#)). It follows the player and never shows space outside the room.

Follow behavior

- The camera follows the player by mapping the player's **reachable range** — the bounding box of the walkable area — onto the camera's full clamped scroll range, per axis. The leftmost/topmost reachable point shows the start of the background; the rightmost/bottommost shows the end. This aims to cover the whole background across a traversal, even though the player can never reach the room's literal edges (the walkable area is usually a strip inside the room).
- That reveal is bounded by an **on-screen clamp**: the scroll never moves so far that the player's pivot leaves the viewport. A margin (default 15% of the viewport per axis) keeps clickable floor around the player. When the walkable area is a thin slice of a much larger room, the raw reveal would scroll the player off screen — or into an unclickable sliver — and trap them; the clamp caps the scroll so the player stays visible, sacrificing edge coverage only as much as needed. Well-proportioned rooms (a walkable strip that is not tiny relative to the room) reach full coverage before the clamp engages. The room bounds remain the hard limit, so near a literal room edge the player may sit at the viewport edge (there is nothing further to reveal).
- The camera is always clamped to the room bounds. A room no larger than the scenery viewport is centered and does not scroll.
- Both axes use the same mapping. A room wider than the scenery viewport scrolls horizontally; a taller room scrolls vertically by the same mechanism. Vertical scrolling is therefore not a separate feature — it falls out of the clamp for tall rooms, which are uncommon.
- When no walkable area is defined, the reachable range defaults to the whole room, degenerating to ordinary clamped follow.

Scripted overrides

Call	Effect
<code>camera_look_at(target)</code>	Snap the camera to the target and suspend follow.
<code>camera_go_to(target)</code>	Tween the camera to the target, yield the task, and suspend follow.
<code>camera_follow_player()</code>	Resume following the player.

While a scripted override is active, automatic follow is suspended. Follow also resumes automatically when the room view returns to `Command` state. See [The scripting API](#) for the camera functions.

Screen edges

`on_screen_edge(edge)` fires when the **player** reaches a room edge — not when the camera does — so edge-triggered room exits work at the end of a scroll or in rooms that do not scroll.

Hotspots

A hotspot is an interactive part of the scenery. It may represent:

- a background region;
- an active object;
- an NPC;
- a purely logical area.

A hotspot has:

Field	Meaning
<code>id</code>	Stable internal identifier.
<code>name</code>	Localized display noun shown in the command bar.
<code>area</code>	Optional explicit hit-test polygon.
<code>bind</code>	Optional binding to a visual object or region.
<code>approach</code>	Point the player walks to before executing a command.
<code>affordances</code>	Verbs that the UI may offer for this hotspot.
<code>default_verb</code>	Optional verb used on a plain click. Must be <code>look_at</code> or in <code>affordances</code> . Defaults to <code>look_at</code> .

At least one of `area` or `bind` is required. If both are present, the hotspot is hit when either the explicit area contains the pointer or the bound visual is hit. This lets authors use a precise polygon, object/region collision, or both.

`id` and `name` are intentionally separate. IDs should be ASCII, stable, and script-friendly. Names may contain spaces, accents, and localized text.

Example:

```
hotspots:
  surface_sublimator:
    name: "aparato para sublimar superficies"
    area: [ ... ]
    approach: at_sublimator
    affordances: [ look_at, use ]
```

Affordances

Affordances declare which verbs make sense to offer for a hotspot. They gate UI selection; they do not guarantee success. The Lua handler decides the outcome.

Every hotspot and inventory item implicitly supports `look_at` even when it is not listed; `affordances` enumerates the *additional* verbs offered. A `default_verb` must therefore be `look_at` or one of the listed affordances.

Affordance	Enables verb(s)
<code>look_at</code>	Look at
<code>talk_to</code>	Talk to
<code>pick_up</code>	Pick up
<code>use</code>	Use as one-operand command or as operand of <code>Use X with Y</code>
<code>give</code>	Give as recipient
<code>open</code>	Open
<code>close</code>	Close
<code>push</code>	Push
<code>pull</code>	Pull

The first implementation shall use direct verb names as affordances. A future version may introduce higher-level categories such as `open_closeable`, but the runtime dispatch still resolves to concrete verbs.

Avatars

An avatar is the visual and spatial representation of a player character or NPC. The room does not depend on the drawing strategy used by the avatar.

Possible drawing strategies:

Strategy	Cast type	Example
Single animated sprite	<code>animated_sprite</code>	One spritesheet with all frames.
Composite sprite	<code>composite</code>	Body + head + props.
Future skeletal/layered renderer	<code>skeletal</code> (design-for)	More advanced animation system.

A character's appearance selects the strategy and supplies its parameters in `cast.yaml`; see [appearance types](#). The room interacts with avatars through a stable interface regardless of the chosen strategy.

Player vs NPC avatars

The player avatar is a persistent world entity: it is created once when `RoomScene` starts and survives every `change_room`. NPC avatars are room-scoped — created on room load and destroyed on room unload. The player avatar is owned at the world / `RoomScene` level; NPC avatars are owned by `RoomRuntime`.

The player character is named by the `RoomScene` manifest entry's `player` parameter (a cast character id). At `RoomScene` start the engine builds the persistent player avatar from that character's cast appearance and seats it in `start_room` using the placement resolution order below.

A room's `avatars` entry with `player: true` does **not** create the player. It only declares where the player is placed when this room is entered without an explicit entry point, and its appearance binding is ignored for the player (it is used only for NPCs). The player's appearance is fixed at game start from the cast character named by `player`; there is no MVP script API to change it mid-game (runtime appearance changes are design-for).

`change_room(id, entry_point?)` resolves the player's placement in order:

1. the `entry_point` argument, if given (a named point in the target room);
2. otherwise the target room's `player: true` avatar `start` point;
3. otherwise the room's `player_start` point;
4. otherwise fail loudly in development builds.

The player's position and orientation at unload are not carried into the next room — the resolved entry point fully determines placement. Carrying position across rooms is a design-for refinement.

Avatar interface

Method	Purpose
<code>set_position(x, y)</code>	Place avatar in room coordinates.
<code>position()</code>	Return avatar's main position, usually walking pivot.
<code>anchor(name)</code>	Return absolute position of a named anchor.
<code>face(direction)</code>	Set orientation.
<code>facing()</code>	Return orientation.
<code>set_scale(scale)</code>	Set visual scale.
<code>stand()</code>	Play standing animation.
<code>walk()</code>	Play walking animation.
<code>talk()</code>	Play talking animation.
<code>play(sequence)</code>	Play named animation sequence.
<code>play_until_end(sequence)</code>	Play non-looping sequence and yield script until done.
<code>move_to(target)</code>	Follow room pathfinding to a point or named target.
<code>draw(target)</code>	Draw avatar.
<code>z()</code>	Return depth key.

This is the full C++ interface. The Lua script handle `avatar(id)` exposes a deliberately smaller subset (`move_to`, `look_at`, `face`, `play`, `play_until_end`, `position`, `anchor`); see [The scripting API](#). Methods such as `set_position`, `stand`, `walk`, `talk`, and `set_scale` are engine-internal and not script-exposed — talking animation is driven by the global `talk`, and standing/walking by movement and the default action mapping below.

Default action mapping:

Command action	Default avatar animation
<code>look_at</code>	Face target, then stand.
<code>talk_to</code>	Talk.
<code>pick_up</code>	<code>pick_up_<direction></code> , fallback to stand.
<code>open</code>	<code>open_<direction></code> , fallback to stand.
<code>close</code>	<code>close_<direction></code> , fallback to stand.
<code>push</code>	<code>push_<direction></code> , fallback to stand.
<code>pull</code>	<code>pull_<direction></code> , fallback to stand.
<code>give</code>	<code>give_<direction></code> , fallback to stand.
<code>use</code>	Context-specific, fallback to stand.

Commands and SCUMM panel

The player builds commands of the form:

<verb> <param1?> <param2?>

Parameters are hotspots or inventory items.

Supported verbs for the first implementation:

Look at	Talk to	Pick up
Use	Give	Open
Close	Push	Pull

`Walk to` is not a verb button. Clicking a walkable floor point moves the player there. Clicking a point **outside** the walkable area (that is not a hotspot) routes the player to the nearest reachable point on the walkable boundary rather than doing nothing. The target uses click-to-move only; keyboard-driven avatar movement from the prototype is not part of the design.

Clicking empty scenery (no hotspot) while a command is being built **cancels** the command and returns the builder to `IDLE`; the same click while already `IDLE` walks the player as above.

Default verb on click

Left-clicking a hotspot or inventory item with no verb currently selected starts a command immediately using that entity's **default verb**:

- if the entity declares `default_verb` in YAML (which must appear in its `affordances`), that verb is used;
- otherwise the default verb is `look_at`.

The command then proceeds through the normal state machine, so a default verb that needs a second operand (for example a `combinable` item under `use`) still advances to `EXPECTING_PARAM2_*`. Left-clicking walkable floor with no verb selected walks the player. Right-click as an "examine" shortcut is a design-for input addition, not part of the MVP.

Command argument types

Type	Meaning
<code>ROOM_OBJECT</code>	Hotspot in the current room, including NPC hotspots.
<code>INVENTORY_OBJECT</code>	Item in the inventory.
<code>ANY_OBJECT</code>	Room object or inventory object.

The implementation may internally distinguish `NPC`, `REGION`, `OBJECT`, and `EXIT`, but the command builder only needs the argument classes above.

Verb argument rules

Verb	Param 1	Param 2	Connector	Notes
look_at	ANY_OBJECT	—	—	Display description or default fallback.
talk_to	ROOM_OBJECT	—	—	Usually requires NPC affordance.
pick_up	ROOM_OBJECT	—	—	Adds item or returns refusal.
open	ANY_OBJECT	—	—	Requires open affordance.
close	ANY_OBJECT	—	—	Requires close affordance.
push	ANY_OBJECT	—	—	Requires push affordance.
pull	ANY_OBJECT	—	—	Requires pull affordance.
use	ANY_OBJECT	optional ANY_OBJECT	with	One-operand or two-operand.
give	INVENTORY_OBJECT	ROOM_OBJECT	to	Handler runs on recipient.

Localized connectors are handled by the UI formatter, not by the command model. Verb labels and connectors are looked up by key in the [UI strings resource](#); they are never hardcoded. For Spanish the strings file supplies, for example:

Verb	Connector
use	con
give	a

Command builder state machine

The command builder exists only in the room-view `Command` state.

States

State	Meaning
IDLE	No command is being built. Hover text may show the hovered verb or object.
VERB_SELECTED	A verb has been selected; the builder chooses the required next parameter state.
EXPECTING_PARAM1_ROOM_OBJECT	Waiting for first argument; room hotspots only.
EXPECTING_PARAM1_INVENTORY_OBJECT	Waiting for first argument; inventory only.
EXPECTING_PARAM1_ANY_OBJECT	Waiting for first argument; room or inventory.
EXPECTING_PARAM2_ROOM_OBJECT	Waiting for second argument; room hotspots only.
EXPECTING_PARAM2_INVENTORY_OBJECT	Waiting for second argument; inventory only.
EXPECTING_PARAM2_ANY_OBJECT	Waiting for second argument; room or inventory.
COMMAND_READY	Command has all required data and can be executed.
COMMAND_EXECUTING	Player walks to approach point and Lua handler runs.

Transition table

Current state	Input	Condition	Next state	Action
IDLE	Click verb	Verb exists	VERB_SELECTED	Store verb.
IDLE	Click walkable floor	No verb selected	COMMAND_EXECUTING	Execute walk command.
IDLE	Click hotspot/inventory	No verb selected	Depends on default verb	Start command with the entity's <code>default_verb</code> , or <code>look_at</code> if none.
VERB_SELECTED	Internal	Verb expects room param 1	EXPECTING_PARAM1_ROOM_OBJECT	—
VERB_SELECTED	Internal	Verb expects inventory param 1	EXPECTING_PARAM1_INVENTORY_OBJECT	—
VERB_SELECTED	Internal	Verb expects any param 1	EXPECTING_PARAM1_ANY_OBJECT	—
EXPECTING_PARAM1_*	Click valid object	Type and affordance match	Depends on verb/object	Store param 1.
EXPECTING_PARAM1_*	Click invalid object	Type or affordance mismatch	Same state	Optional feedback.
EXPECTING_PARAM1_*	Click another verb	—	VERB_SELECTED	Replace verb and clear params.
EXPECTING_PARAM1_*	Cancel	—	IDLE	Clear command.
EXPECTING_PARAM2_*	Click valid object	Type and affordance match	COMMAND_READY	Store param 2.
EXPECTING_PARAM2_*	Click invalid object	Type or affordance mismatch	Same state	Optional feedback.
EXPECTING_PARAM2_*	Click another verb	—	VERB_SELECTED	Replace command.
EXPECTING_PARAM2_*	Cancel	—	IDLE	Clear command.
COMMAND_READY	Internal	—	COMMAND_EXECUTING	Dispatch command.
COMMAND_EXECUTING	Command finished	—	IDLE	Clear command and restore UI.

The **Cancel** input is the player clicking empty scenery (a point that is neither a hotspot nor the panel) while a command is being built.

Verb-to-state mapping

Verb	After verb selected	After param 1
look_at	EXPECTING_PARAM1_ANY_OBJECT	COMMAND_READY
talk_to	EXPECTING_PARAM1_ROOM_OBJECT	COMMAND_READY
pick_up	EXPECTING_PARAM1_ROOM_OBJECT	COMMAND_READY
open	EXPECTING_PARAM1_ANY_OBJECT	COMMAND_READY
close	EXPECTING_PARAM1_ANY_OBJECT	COMMAND_READY
push	EXPECTING_PARAM1_ANY_OBJECT	COMMAND_READY
pull	EXPECTING_PARAM1_ANY_OBJECT	COMMAND_READY
use	EXPECTING_PARAM1_ANY_OBJECT	COMMAND_READY or EXPECTING_PARAM2_ANY_OBJECT
give	EXPECTING_PARAM1_INVENTORY_OBJECT	EXPECTING_PARAM2_ROOM_OBJECT

use disambiguation

`use` may be immediate or combinational. Only an inventory item can be the first operand of a two-operand `use`; a room hotspot as the first operand is always one-operand (`Use drawer`). To combine, the inventory item is the first operand and the room hotspot or other item is the second (`Use key with drawer`).

Param 1	Condition	Next state
Room hotspot	always	COMMAND_READY
Inventory item	<code>combinable: false</code>	COMMAND_READY
Inventory item	<code>combinable: true</code>	EXPECTING_PARAM2_ANY_OBJECT

Because only inventory items carry the `combinable` flag, the builder never needs a one-vs-two-operand signal on hotspots. For the MVP, `combinable` is an acceptable inventory-item flag. Later versions may replace it with a richer rule system where an item may be immediate, combinable, or both depending on context.

Command model

The command builder produces a language-independent command object.

```
struct Command {
    Verb verb;
    ObjectRef param1;
    std::optional<ObjectRef> param2;
};
```

Script-friendly form:

```
{
  verb = "use",
  param1 = "lighter",
  param2 = "wood"
}
```

The UI formats this command into readable text using localized verb labels, connectors, and object display names.

Top bar behavior

The command bar previews the command currently being built.

Command state	Hovered element	Display text example
IDLE	Nothing	Empty string or default prompt.
IDLE	Verb	Use
IDLE	Room object	drawer / localized name.
IDLE	Inventory item	key / localized name.
IDLE	Walkable floor	The walk label (Ir a / localized), from the UI strings resource .
EXPECTING_PARAM1_*	Nothing	Use
EXPECTING_PARAM1_*	Valid object	Use key
EXPECTING_PARAM1_*	Invalid object	Use
EXPECTING_PARAM2_*	Nothing	Use key with
EXPECTING_PARAM2_*	Valid object	Use key with door
EXPECTING_PARAM2_*	Invalid object	Use key with
COMMAND_EXECUTING	Any	Last command text, optionally styled as executing.

The top bar shall not build logic from strings. It formats the internal command state for the player.

Building and running a command

When a command becomes ready:

1. The room view switches to `COMMAND_EXECUTING` within the command builder.
2. If the command target has an `approach` point, the player avatar walks there. An `approach` point should lie inside the walkable area; if it does not, the engine routes the avatar to the

- nearest walkable point and, in development builds, logs a warning.
3. The avatar faces the target when appropriate.
 4. The engine dispatches the command to the Lua handler.
 5. The handler may run scripted actions, change state, start a dialog, change a room, or return a caption.
 6. When the command finishes, the command builder returns to `IDLE`.

For two-operand verbs, inventory-item behavior has priority when the first operand is an inventory item and `inventory.lua` defines a matching handler. Otherwise the handler is invoked on the second operand.

Command	Handler
Use key with door, with <code>inventory.key.use</code> defined	<code>inventory.key.use("door")</code>
Use key with door, without <code>inventory.key.use</code>	<code>door.use("key")</code>
Give map to Stan, with <code>inventory.map.give</code> defined	<code>inventory.map.give("stan")</code>
Give map to Stan, without <code>inventory.map.give</code>	<code>stan.give("map")</code>

Dialog system

A dialog is a tree of NPC lines and player choices. Dialogs are written in Lua so nodes can run script code, branch on conditions, set state, move characters, or give items.

A dialog is started with:

```
start_dialog("stan")
```

The dialog id is also the **speaking NPC's character id**: it selects the dialog file (`dialogs/stan.lua`) and the cast character whose speech color and position render the node `npc` lines. The dialog file therefore declares no speaker field.

While a dialog runs, the room view enters `Dialog` state. The SCUMM panel is replaced by dialog options. When the dialog ends, the room view returns to `Command` state.

Dialog format

```
return {
  start = "greet",

  greet = {
    npc = "¿Buscás un barco?",
    options = {
      { "Contame más.", to = "ships" },
      { "¿Cuánto cuesta?", to = "price", once = true },
      { "Chau.", to = END },
    },
  },

  ships = {
    npc = "Los mejores del lago.",
    to = "greet",
  },

  price = {
    npc = "Cinco mil piezas de ocho.",
    options = {
      {
        "Trato hecho.",
        when = function() return has_item("gold") end,
        run = function()
          remove_item("gold")
          add_item("ship")
        end,
        to = END,
      },
      { "Muy caro.", to = "greet" },
    },
  },
}
```

Field	Applies to	Meaning
start	Dialog	Entry node id.
on_enter	Dialog	Optional setup callback.
on_exit	Dialog	Optional cleanup callback.
npc	Node	NPC line or list of lines.
options	Node	Player choices.
to	Node or option	Next node id or <code>END</code> .
when	Option	Visibility predicate, re-evaluated every time the node's options are shown . Return false to hide the option until its condition holds.
run	Option	Code executed when selected.
once	Option	Option is hidden permanently after it is used. Consumption is persisted (survives leaving and restarting the dialog, and save/load).
silent	Option	Player line is not spoken aloud.

Dialog execution

1. Speak the node's `npc` line or lines.
2. If the node has options, display available options whose `when` passes.
3. Wait for player selection.
4. Speak the selected player line unless `silent` is true.
5. Run the option's `run` callback, if present.
6. Consume `once` options.
7. Follow `to`.
8. End when `to == END`.

A node declares **either** `options` **or** a `to`, never both (the loader rejects a node that has both). When an options node's choices are all hidden — every one consumed by `once` or filtered out by a failing `when` — it has nothing left to offer and the dialog ends. This is what lets a conversation "run dry" gracefully once everything worth saying has been said; route the player somewhere instead by keeping a always-available fallback option (e.g. a "Chau." line `to = END` or back to a hub node).

Evolving dialogs

A conversation is not a fixed menu: `once`, `when`, and the state stores together let the option list **prune and unlock as the player learns things and the world changes**. Three building blocks:

- **Ask-once lines.** Mark an option `once = true` so a question like "¿Quién sos?" appears the first time and never again. Consumption is persisted under an engine-reserved `__dialog`.

`<npc>.<node>.<index>` key in global state (it folds into `GameState`), so it survives leaving the dialog, re-entering it, and save/load — the player never re-asks a settled question.

- **Conditional unlocking.** Gate an option with `when = function() ... end`. The predicate is checked each time the node's options are displayed, so an option can appear only once a fact is known or an item is held, and a path can be closed off by the same mechanism. Read the engine state stores from `when / run (get_state, get_room_state, has_item)` and write them from `run / node callbacks (set_state, ...)` — never from Lua locals, which are not persisted.
- **Branch reshaping.** Combine the two: a `run` callback sets a state flag, and other options' `when` predicates read it, so choosing one line enables or disables others on the next pass through the node.

```
return {
  start = "greet",

  greet = {
    npc = "¿Y vos quién sos?",
    options = {
      -- Asked at most once, ever (persisted across save/load).
      { "Soy Julia.", to = "greet", once = true,
        run = function() set_state("skull.met_julia", true) end },

      -- Appears only after we've introduced ourselves.
      { "¿Conociste a mi abuelo?", to = "grandfather",
        when = function() return get_state("skull.met_julia") end },

      -- Appears only once the lever puzzle is solved, then never repeats.
      { "El mecanismo ya está abierto.", to = "reward", once = true,
        when = function() return get_room_state("lab.lever_pulled") end },

      { "Nada, me voy.", to = END },
    },
  },

  grandfather = {
    npc = "Lo recuerdo. Dejó algo para vos.",
    run = function() set_state("skull.grandfather_topic", true) end,
    to = "greet",
  },

  reward = {
    npc = "Tomá esto.",
    run = function() add_item("amulet") end,
    to = "greet",
  },
}
```

On the first visit only "Soy Julia." and "Nada, me voy." are offered. After the introduction the grandfather line unlocks; once the lab lever is pulled the mechanism line appears (and disappears after use). Because "Nada, me voy." has no `when` or `once` it is always available, so `greet` never runs dry and the player can always leave — drop that fallback (or gate it) and the node would instead end the dialog on its own once every option is consumed or filtered out.

Speech

Speech is rendered over the scenery near the speaker. The speech font is the `RoomScene font` parameter (a logical path; an engine default applies when omitted); per-character speech color and style come from the cast file.

Every spoken line shall have a stable id or be representable by one. This enables future voice-over attachment.

Speech rendering supports:

- wrapping;
- configurable duration;
- skipping;
- per-character color;
- optional positioning override.

When `talk` is called without an explicit duration, the engine computes one from the line length:

```
duration = clamp(1.0 s, 0.5 s + 0.06 s × character_count, 7.0 s) / text_speed
```

`text_speed` is the player text-speed setting (default 1.0; higher is faster). The computed duration is an upper bound — speech is always skippable, so the player may advance sooner. When a voice-over clip is attached to a line, the clip length overrides this estimate.

Characters and movement

Characters move through room geometry. A destination must be inside the walkable area and outside all obstacles. The pathfinder returns waypoints, and the avatar walks them segment by segment.

Characters scale with the room's perspective function as they move. Their z-order is normally their walking pivot y coordinate.

Inventory

Inventory items are command parameters. They have ids, localized names, optional descriptions, and affordances.

Inventory uses the same static-data / behavior split as rooms:

File	Purpose
<code>inventory.yaml</code>	Static item definitions: names, descriptions, affordances, default verbs, optional future icons.
<code>inventory.lua</code>	Item verb handlers and shared inventory behavior.

Example `inventory.yaml`:

```
items:
  key:
    name: "la llave"
    affordances: [ look_at, use ]
    combinable: true
```

Example `inventory.lua`:

```
local inventory = {}

inventory.key = {
  look_at = function()
    return "Una llave pequeña."
  end,

  use = function(target)
    if target == "door" then
      remove_item("key")
      change_room("hall")
      return "La llave funciona."
    end
    return "No parece servir."
  end,
}

return inventory
```

The MVP displays inventory items as text using their localized `name`. Each item may later declare an optional `icon` — a spritesheet frame id or image logical path — which the SCUMM panel draws instead of text. Inventory items support the same optional `default_verb` as hotspots: it must be one of the item's `affordances` and defaults to `look_at`.

Migration notes

Prototype divergences and refactoring tasks are tracked separately in [migration notes](#). They are not part of the target design.

The scripting API

A game is static data plus behavior:

```
YAML = structure, assets, geometry, appearances, affordances
Lua  = behavior, rules, dialog branching, scripted actions
```

Lua runs as cooperative coroutines. Blocking-looking calls such as `wait`, `talk`, `avatar(id):move_to`, and `avatar(id):play_until_end` yield to the engine loop instead of freezing the frame.

Game wiring

A standard game is organized as:

<code>game.yaml</code>	manifest: engine config + scene list + entry scene
<code>strings/</code>	
<code>es.yaml</code>	engine-emitted UI strings (verbs, connectors, menus)
<code>cast.yaml</code>	appearances + characters
<code>game.lua</code>	global logic and shared fallbacks
<code>inventory.yaml</code>	static inventory item definitions
<code>inventory.lua</code>	inventory item behavior
<code>rooms/</code>	
<code>study.yaml</code>	static room definition
<code>study.lua</code>	room behavior
<code>dialogs/</code>	
<code>stan.lua</code>	dialog tree

The manifest declares the top-level scenes. Its `RoomScene` entry points at the game cast, global logic file, inventory files, room directory, start room, and the cast character id that is the player.

Example manifest excerpt:

```
scenes:
- id: room_view
  type: RoomScene
  parameters:
    cast: cast.yaml
    logic: game.lua
    inventory: inventory.yaml
    inventory_logic: inventory.lua
    rooms: rooms
    start_room: study
    player: julia
```

Room ids resolve to files:

```
change_room("study") -> rooms/study.yaml + rooms/study.lua
```

Dialog ids resolve to files, and the same id names the speaking NPC character:

```
start_dialog("stan") -> dialogs/stan.lua, spoken by cast character "stan"
```

Cast and appearances

Cast is static data and lives in YAML.

```
appearances:
  julia_field:
    type: composite                                # CompositeSprite (gfx layer)
    composite: anims/julia.composite.yaml
    shadow:
      size: { x: 70, y: 18 }
      color: { r: 0, g: 0, b: 0, a: 70 }

  guard:
    type: animated_sprite                          # single AnimatedSprite (gfx layer)
    sprite: anims/guard.anim.yaml
    shadow:
      size: { x: 60, y: 16 }
      color: { r: 0, g: 0, b: 0, a: 70 }

characters:
  julia:
    appearance: julia_field
    name: "Julia"
    speech_color: { r: 255, g: 255, b: 180 }

  schneider:
    appearance: julia_field
    name: "Dra. Schneider"
    speech_color: { r: 120, g: 220, b: 255 }
```

Section	Meaning
appearances	Reusable visual definitions.
characters	Character ids, display names, speech style, and appearance binding.

Character ids are stable script identifiers. Display names are localized text.

Appearance types

An appearance selects which avatar drawing strategy implements it and supplies that type's parameters. The avatar interface is identical regardless of type (see the avatar strategies in [Third-person point & click concepts](#)); only construction differs.

type	Parameters	Backed by	Scope
animated_sprite	sprite: an *.anim.yaml	pac::gfx::AnimatedSprite	MVP
composite	composite: a *.composite.yaml	pac::gfx::CompositeSprite	MVP
skeletal	renderer-specific	future layered/skeletal renderer	Design-for

`shadow` is a common, optional appearance field handled by the point-and-click layer for every type. The animated-sprite and composite file formats are specified in [2D game concepts](#).

Room scripts

A room script returns a table containing lifecycle hooks and hotspot verb handlers.

```
local room = {}

function room.on_load()
    play_music("music/study.ogg", true)
end

function room.on_unload()
    stop_music()
end

room.on_zone_enter = function(zone)
    if zone == "to_hall" then
        change_room("hall")
    end
end

room.hotspots = {
    drawer = {
        look_at = function()
            return "Es un cajón viejo."
        end,

        open = function()
            set_region_state("drawer", "open")
            set_room_state("drawer.open", true)
            return "Lo abro."
        end,
    },

    door = {
        use = function(item)
            if item == "key" then
                remove_item("key")
                change_room("hall")
                return "La llave funciona."
            end
            return "No parece servir."
        end,
    },
}

return room
```

Room lifecycle hooks

Hook	Called when
<code>on_load()</code>	Room has been loaded and initialized.
<code>on_unload()</code>	Room is about to be unloaded.
<code>on_screen_edge(edge)</code>	Player reaches a room screen edge (left / right / top / bottom); all four fire.
<code>on_zone_enter(zone)</code>	Player enters a zone polygon.
<code>on_zone_exit(zone)</code>	Player exits a zone polygon.

Hotspot handler signatures

Verb kind	Example command	Handler location	Signature
Single operand	Look at drawer	Target hotspot	<code>function()</code>
Single operand	Open drawer	Target hotspot	<code>function()</code>
Two operands: use	Use key with door	Second operand, if no inventory handler exists	<code>function(first_id)</code>
Two operands: give	Give map to Stan	Second operand, if no inventory handler exists	<code>function(first_id)</code>

A per-hotspot handler is bound to its own target, so it receives only the operands it does not already own: nothing for a single-operand verb, and the other operand's id for a two-operand verb. Generic fallbacks in `game.lua` receive the full operand list instead (see the Global game logic section).

A handler may:

- return a caption string;
- call engine API functions;
- start a dialog;
- move avatars;
- change room or global state;
- spawn longer scripted tasks.

If a handler returns a string, the engine displays it as speech or as configured for command feedback.

Global game logic

`game.lua` contains shared behavior that is not local to one room:

- default verb fallbacks;
- global puzzle rules;
- helper functions;
- shared events;
- world-state initialization.

Example:

```

local game = {}

game.fallbacks = {
    look_at = function(target)
        return "No veo nada especial."
    end,

    use = function(a, b)
        return "No funciona."
    end,
}

function game.on_start()
    set_state("mummy.awake", false)
end

return game

```

If a hotspot does not provide a specific handler for a valid command, the command system may fall back to `game.lua`.

Inventory scripts

Inventory items are declared in `inventory.yaml`; their behavior lives in `inventory.lua`.

```

local inventory = {}

inventory.key = {
    look_at = function()
        return "Una llave pequeña."
    end,

    use = function(target)
        if target == "door" then
            remove_item("key")
            change_room("hall")
            return "La llave funciona."
        end
        return "No parece servir."
    end,
}

return inventory

```

Inventory handlers follow the same operand rule as hotspot handlers: the handler is bound to its own item, so it receives only the operands it does not already own. A one-operand `look_at` handler receives nothing. A two-operand `use` or `give` handler on the first item receives the second object's id.

For two-operand commands whose first operand is an inventory item, the dispatcher checks `inventory.lua` first. If the item has a matching handler, it runs that handler. Otherwise dispatch falls back to the second operand's hotspot handler, then to `game.lua` fallbacks.

Dialog scripts

A dialog script returns a node table.

```
return {
  start = "greet",

  greet = {
    npc = "¿Necesitás algo?",
    options = {
      { "¿Quién sos?", to = "who" },
      { "Nada.", to = END },
    },
  },

  who = {
    npc = "Soy la Dra. Schneider.",
    to = "greet",
  },
}
```

Dialog callbacks may use the same API as room scripts.

Coroutine rules

Blocking calls must run inside a spawned task or inside an engine-invoked script context that is already coroutine-enabled.

Valid:

```
spawn(function()
  talk("julia", "Voy hasta la puerta.")
  avatar("julia"):move_to("at_door")
end)
```

Invalid in plain initialization code:

```
-- Do not block during module loading.
talk("julia", "This should not run at load time.")
```

`spawn(fn)` inherits the current script scope. A task spawned from a room hook or hotspot handler is cancelled when that room unloads or reloads; a task spawned from a dialog callback is cancelled when the dialog exits; a task spawned from a cutscene script is cancelled when that scene finishes, is skipped, or is replaced.

Cancelled tasks are removed from the scheduler and are never resumed. Cancellation does not run Lua cleanup code; use `on_unload` for room cleanup and dialog `on_exit` for dialog cleanup. Running coroutines are transient and are not saved. On load-game, the engine cancels the current script scopes, restores `GameState`, loads the restored room, and runs the normal `on_load`.

Cross-room background tasks are design-for. If the API later adds `spawn_global(fn)`, it must be used deliberately and those tasks still will not be part of save/load.

API reference

Flow and coroutines

Function	Parameters	Returns	Meaning
<code>spawn(fn)</code>	function	task id or nil	Run function as coroutine task in the current script scope.
<code>wait(seconds)</code>	number	—	Yield task until time elapses.
<code>emit(name, payload?)</code>	string, optional value	—	Emit named event within the emitter's scope.
<code>wait_event(name)</code>	string	payload	Yield until named event occurs in the task's scope.

Events are scoped: `emit` delivers only to waiters in the **same** script scope as the emitter (room, dialog, scene, or global). Events do not cross scopes, so a later room cannot resume a task that was waiting in an earlier one.

Speech and dialog

Function	Parameters	Returns	Meaning
<code>talk(speaker, text, pos?, dur?)</code>	character id, string, optional position, optional duration	—	Show speech near the speaker and yield until finished/skipped.
<code>show_text(text, dur?)</code>	string, optional duration	—	Show a speaker-less centered text page in virtual space and yield until finished/skipped.
<code>clear_text()</code>	—	—	Remove the current text page immediately.
<code>start_dialog(id)</code>	dialog id	—	Enter dialog state and run dialog.

`text` may later be replaced or supplemented by a stable line id for localization and voice-over. When `dur` is omitted, the engine derives the display time from the line length (see [Speech](#)); `talk` and `show_text` share that formula and are both skippable.

`talk` is the in-room, near-speaker primitive; `show_text` is the speaker-less page primitive used by `StoryText` cutscenes (and any script that wants a centered page). Each `show_text` replaces the current page — a script calls it in sequence, each call yielding until the page finishes or is skipped.

Rooms and camera

Function	Parameters	Returns	Meaning
<code>change_room(id, entry_point?)</code>	room id, optional point id	—	Load another room inside <code>RoomScene</code> .
<code>current_room()</code>	—	room id	Return current room id.
<code>camera_look_at(target)</code>	target	—	Snap camera to target and suspend follow.
<code>camera_go_to(target)</code>	target	—	Tween camera to target, yield until done, and suspend follow.
<code>camera_follow_player()</code>	—	—	Resume following the player.

A target may be:

- a named point;
- an avatar id;
- `{ x = ..., y = ... }` coordinates.

`change_room` reseats the persistent player avatar rather than recreating it; `entry_point` overrides the target room's default placement. See [player vs NPC avatars](#) for the full placement resolution order.

Avatars

`avatar(id)` returns an avatar handle.

Method	Parameters	Returns	Meaning
<code>:move_to(target)</code>	target	—	Move through room pathfinding and yield until done.
<code>:look_at(target)</code>	target	—	Face target.
<code>:face(direction)</code>	"up", "right", "down", "left"	—	Set orientation.
<code>:play(sequence)</code>	string	—	Play animation sequence.
<code>:play_until_end(sequence)</code>	string	—	Play non-looping animation and yield until done.
<code>:position()</code>	—	point	Return current position.
<code>:anchor(name)</code>	string	point	Return absolute anchor position.

`:look_at(target)` is a convenience that resolves a world target to the nearest of the four orientations and faces it — equivalent to computing a direction and calling `:face`. There is no `:talk()` in the script API: the talking animation is started automatically by the global `talk(speaker, ...)`. Other C++ avatar methods (`set_position`, `stand`, `walk`, `set_scale`) are engine-internal and intentionally not script-exposed.

State

Function	Parameters	Returns	Meaning
<code>get_state(key)</code>	string	value	Read global saved state.
<code>set_state(key, value)</code>	string, value	—	Write global saved state.
<code>get_room_state(key)</code>	string	value	Read current-room saved state.
<code>set_room_state(key, value)</code>	string, value	—	Write current-room saved state.

State keys use dotted names by convention:

```
set_state("mummy.awake", true)
set_room_state("drawer.open", true)
```

`get_room_state` / `set_room_state` operate only on the **current** room's store; they cannot read or write another room. Facts that must be visible across rooms belong in global state under a namespaced key (for example `set_state("hall.lever_pulled", true)`).

State values are restricted to **scalars — boolean, number, or string** — for the MVP. Tables and nested values are out of scope so the save file stays simple and serialization is total; storing an unsupported type fails loudly in development builds. Region states are not stored here — they are managed by `set_region_state` / `get_region_state` (see Scenery below).

Scenery

Function	Parameters	Returns	Meaning
<code>set_region_state(id, state)</code>	region id, state id	—	Swap a background region to a named state.
<code>get_region_state(id)</code>	region id	state id	Read a region's current state.
<code>show_object(id)</code>	object id	—	Make room object visible/enabled.
<code>hide_object(id)</code>	object id	—	Hide room object and disable interaction if bound.
<code>set_layer_visible(id, visible)</code>	layer id, bool	—	Show/hide a background layer (the layer must have an <code>id</code>). Persisted per room.
<code>enable_hotspot(id)</code>	hotspot id	—	Allow hotspot interaction.
<code>disable_hotspot(id)</code>	hotspot id	—	Prevent hotspot interaction.

Driving an object's own animation from a script (`object(id):play(...)`) is a design-for addition; the MVP only shows or hides objects.

Inventory

Function	Parameters	Returns	Meaning
<code>has_item(id)</code>	item id	bool	Whether player has item.
<code>add_item(id)</code>	item id	—	Add item to inventory.
<code>remove_item(id)</code>	item id	—	Remove item from inventory.
<code>list_items()</code>	—	list	Return current inventory ids.

Audio

Function	Parameters	Returns	Meaning
<code>play_music(path, loop?)</code>	logical path, optional bool	—	Start streamed music.
<code>stop_music()</code>	—	—	Stop current music.
<code>play_sound(path)</code>	logical path	—	Play short sound effect.
<code>stop_sounds()</code>	—	—	Stop all active sound effects.

Input and room-view control

Function	Parameters	Returns	Meaning
<code>block_input()</code>	—	—	Switch room view to blocked input mode.
<code>unblock_input()</code>	—	—	Restore command input mode.
<code>set_room_view_state(state)</code>	string	—	Explicitly set room-view state when needed.

Scripts should prefer high-level operations such as `start_dialog` and command execution, which manage room-view state automatically.

Resources

Function	Parameters	Returns	Meaning
<code>resource_path(rel)</code>	logical path	logical resource id	Normalize or validate a logical resource path.

`resource_path` does not return a platform filesystem path. It returns a logical resource id suitable for the engine resource layer.

Error handling

Script API calls shall fail loudly during development when called with invalid ids or malformed parameters. In release builds, failures should produce safe fallback behavior when possible and log diagnostics.

Examples:

Error	Development behavior	Release behavior
Missing hotspot handler	Log clear error and use fallback if available.	Use fallback response.
Unknown room id	Error and refuse transition.	Error and remain in current room.
Unknown animation sequence	Log error, fallback to <code>stand</code> .	Fallback to <code>stand</code> .
Invalid resource path	Error with logical path.	Log and skip asset/action.

Template scripts

The engine shall provide templates for:

- `game.yaml`;
- `cast.yaml`;
- `game.lua`;
- `rooms/<id>.yaml`;
- `rooms/<id>.lua`;
- `dialogs/<id>.lua`.

Templates encode the expected behavior and reduce the amount of boilerplate required to start a new game.

Data formats

Complete field reference for every YAML and Lua data file the engine reads. The worked examples live in 02 – 05 ; this appendix is the exhaustive required / optional / default reference.

Type notation: `{a, b}` is a mapping with those keys, `[T]` is a list of `T`, `polygon` is `[{x, y}, ...]`, `path` is a logical resource path relative to `resources.src`.

Conventions

- All engine YAML data files accept an optional integer `version:` (default 1), bumped when the format changes incompatibly.
- Internal `ids` are ASCII, stable, and script-friendly; display `names` may contain spaces, accents, and localized text.
- Resource references are logical paths, never platform filesystem paths.
- State keys use dotted names; state values are scalars (boolean, number, string).

Game manifest — `game.yaml`

Worked example: [02 — Architecture overview](#).

Field	Req	Type	Default	Meaning
version	opt	int	1	Data-format version.
id	req	string	—	Stable game id used as the per-game subdirectory under the per-user data path (e.g. <code>~/.local/share/<id>/saves/</code>). Must match <code>[a-z0-9_-]+</code> . Two games using this engine get separate save folders by choosing distinct ids.
resolution	req	{width, height}	—	Virtual design resolution.
window	req	{fullscreen, width, height}	—	Initial display mode. <code>fullscreen: true</code> starts the game fullscreen at the optimal mode for the resolution (the video mode matching <code>resolution</code> when available, else the smallest larger mode, letterboxed). <code>width / height</code> are the initial windowed client size (default to <code>resolution</code>). Player settings override these at runtime via the settings scene.
resources	req	{src}	—	Resource source root. <code>src</code> is a directory (MVP) or archive (design-for).
strings	req	path	—	UI strings resource (engine-emitted text). One file in the MVP; a language→file map is design-for. See UI strings .
settings	opt	map	—	Default player-facing settings (e.g. <code>audio.music_volume</code> , <code>audio.sfx_volume</code>). User settings override these.
development	opt	map	—	Dev-only flags: <code>edit_mode</code> , <code>show_walkboxes</code> , <code>show_hotspots</code> , <code>allow_room_reload</code> . Not persisted as player settings.
entry	req	scene id	—	Initial scene.
scenes	req	[scene]	—	Scene list and outcome wiring.

scene entry:

Field	Req	Type	Default	Meaning
id	req	string	—	Scene id, referenced by <code>entry</code> and outcomes.
type	req	enum	—	<code>TitleScreen</code> , <code>StoryText</code> , <code>RoomScene</code> , or a registered custom type.
parameters	opt	map	—	Type-specific parameters (see below).

Scene parameters by type:

- `TitleScreen` — `background` (opt path), `music` (opt path), `new_game` (outcome scene id), `exit` (outcome, often `QUIT`), `font` (opt path).
- `StoryText` / `Cutscene` — `script` (path), `on_finish` (outcome scene id), `font` (opt path).
- `RoomScene` — `cast` (path), `logic` (path), `inventory` (path), `inventory_logic` (path), `rooms` (directory path), `start_room` (room id), `player` (req, cast character id — the persistent player avatar; appearance comes from this character's cast entry), `font` (opt path — speech and panel font).

Scenes that render text take their `font` as a logical-path parameter; an engine default is used when omitted. Per-character speech color and style come from the cast file, not the scene `font`.

UI strings — `strings/<lang>.yaml`

The strings resource holds every user-facing string the engine itself generates: verb labels, command connectors, and built-in menu labels. It does **not** hold game content strings — hotspot/item `names`, speech, and dialog lines stay inline in their own files. The MVP ships one file (Spanish); a language→file map and a runtime selector are design-for (R3).

Field	Req	Type	Default	Meaning
<code>version</code>	opt	int	1	Data-format version.
<code>language</code>	req	string	—	Language tag, e.g. <code>es</code> .
<code>verbs</code>	req	map verb id → string	—	Display label per verb. Keys are the verb ids: <code>look_at</code> , <code>talk_to</code> , <code>pick_up</code> , <code>use</code> , <code>give</code> , <code>open</code> , <code>close</code> , <code>push</code> , <code>pull</code> .
<code>connectors</code>	req	map verb id → string	—	Two-operand connector per verb: <code>use</code> (e.g. <code>con</code>), <code>give</code> (e.g. <code>a</code>).
<code>ui</code>	req	map key → string	—	Built-in UI labels: menu (<code>new_game</code> , <code>continue</code> , <code>settings</code> , <code>quit</code>) and the top-bar walk label <code>walk_to</code> (shown when hovering walkable floor).

```

version: 1
language: es

verbs:
  look_at: "Mirar"
  talk_to: "Hablar con"
  pick_up: "Agarrar"
  use:     "Usar"
  give:    "Dar"
  open:    "Abrir"
  close:   "Cerrar"
  push:    "Empujar"
  pull:    "Tirar"

connectors:
  use: "con"
  give: "a"

ui:
  walk_to: "Ir a"
  new_game: "Nuevo juego"
  continue: "Continuar"
  settings: "Opciones"
  quit:     "Salir"

```

Cast — `cast.yaml`

Worked example: [05 — Scripting API](#).

appearances — map of appearance id → appearance:

Field	Req	Type	Default	Meaning
<code>type</code>	req	enum	—	<code>animated_sprite</code> , <code>composite</code> , or <code>skeletal</code> (design-for).
<code>sprite</code>	req if <code>animated_sprite</code>	path	—	An <code>*.anim.yaml</code> .
<code>composite</code>	req if <code>composite</code>	path	—	A <code>*.composite.yaml</code> .
<code>shadow</code>	opt	<code>{size: {x, y}, color: {r, g, b, a}}</code>	none	Ground shadow drawn under the avatar.

characters — map of character id → character:

Field	Req	Type	Default	Meaning
<code>appearance</code>	req	appearance id	—	Visual definition from <code>appearances</code> .
<code>name</code>	req	string	—	Localized display name.
<code>speech_color</code>	opt	<code>{r, g, b}</code>	engine default	Speech text color.

Room — `rooms/<id>.yaml`

Worked example: [04 — Point & click concepts](#).

Field	Req	Type	Default	Meaning
<code>version</code>	opt	int	1	Data-format version.
<code>id</code>	req	string	—	Room id (matches the file name).
<code>background</code>	req	<code>{color?, layers}</code>	—	See below.
<code>perspective</code>	opt	<code>{top: {y, scale}, bottom: {y, scale}}</code>	base scale	Avatar render scale interpolated by walking-pivot y, clamped outside <code>[top.y, bottom.y]</code> . Omitted $\Rightarrow$ each avatar keeps its base scale. See 04 § Perspective scaling .
<code>walkable</code>	req	polygon	—	Navigable area.
<code>obstacles</code>	opt	<code>[polygon]</code>	<code>[]</code>	Non-walkable polygons inside <code>walkable</code> .
<code>points</code>	opt	map id $\rightarrow$ <code>{x, y}</code>	—	Named coordinates (starts, approach, camera targets).
<code>zones</code>	opt	<code>[{id, polygon}]</code>	—	Trigger polygons for exits / scripted events.
<code>regions</code>	opt	map	—	Changeable background parts (see below).
<code>objects</code>	opt	map	—	Active sprites placed in the room (see below).
<code>hotspots</code>	opt	map	—	Interactive areas + verb handlers (see below).
<code>avatars</code>	opt	<code>[avatar]</code>	—	Player placement + NPCs present at load (see below).

`background.layers` — `[layer]`:

Field	Req	Type	Default	Meaning
id	req	string	—	Layer id.
image	req	path	—	Layer image.
z	req	number	—	Draw depth; larger is nearer the camera.
origin	opt	{x, y}	{0, 0}	Room-space top-left of the layer image (layers may differ in size and be placed freely). The room's world bounds are the union of all layer rects, floored to the room view; see 04 § World bounds .
scale	opt	number	1.0	Uniform render scale about <code>origin</code> , aspect always preserved (never distorted). A development aid for sizing furniture-style occluders; production layers ship native (1.0). The room editor resizes about the layer's base (bottom-centre) and can set <code>z</code> to the scaled base line.
interactive	opt	bool	false	Whether the layer receives pointer interaction.
visible	opt	bool	true	Initial visibility. Toggle at runtime with <code>set_layer_visible(id, bool)</code> (needs an <code>id</code>); persisted per room.
shader	opt	shader ref	—	Design-for.
animation	opt	anim ref	—	Design-for.

`background.color ( {r, g, b, a} )` is an optional solid fill behind all layers.

regions — map of region id → region:

Field	Req	Type	Default	Meaning
area	req	polygon	—	Region footprint.
z	req unless over	number	—	Draw depth.
over	opt	layer id	—	Inherit the named layer's <code>z</code> instead of an explicit <code>z</code> .
states	req	map state id → path	—	Image per named state.
initial	req	state id	—	Starting state.

objects — map of object id → object:

Field	Req	Type	Default	Meaning
sprite	req	path	—	Object animation/sprite.
position	req	{x, y}	—	World position.
z	opt	auto number	auto	auto = world-y of the sprite pivot; a number overrides.
visible	opt	bool	true	Initial visibility; show_object / hide_object change it at runtime.

hotspots — map of hotspot id → hotspot:

Field	Req	Type	Default	Meaning
name	req	string	—	Localized noun shown in the command bar.
area	req if no bind	polygon	—	Explicit hit-test polygon.
bind	req if no area	object:<id> / region:<id>	—	Hit test using the bound visual.
approach	opt	point id {x, y}	—	Where the player walks before the command runs.
affordances	req	[verb]	—	Verbs the UI may offer.
default_verb	opt	verb	look_at	Verb used on a plain click; must be look_at or in affordances.
enabled	opt	bool	true	Initial interactivity; enable_hotspot / disable_hotspot change it at runtime.

At least one of `area` or `bind` is required. If both are present, either hit source activates the hotspot.

avatars — [avatar]:

Field	Req	Type	Default	Meaning
id	req	character id	—	Character to place.
start	req	point id	—	Placement point.
orientation	opt	up / right / down / left	down	Initial facing.
player	opt	bool	false	Marks the player's placement entry (does not create the player; see player vs NPC avatars).

Spritesheet — `<name>.yaml`

Worked example: [03 — 2D game concepts](#).

Field	Req	Type	Default	Meaning
version	opt	int	1	Data-format version.
image	req	path	—	Atlas texture.
size	req	{width, height}	—	Atlas dimensions.
sprites	req	[frame]	—	Named frames.

frame :

Field	Req	Type	Default	Meaning
id	req	string	—	Stable frame id.
rect	req	{x, y, width, height}	—	Rectangle in the atlas.
source_rect	opt	{x, y, width, height}	—	Pre-pack source rect (traceability only).
anchors	opt	map name → {x, y}	—	Frame-local pivot/attachment points (origin = frame top-left).

Animation — `<name>.anim.yaml`

Worked example: [03 — 2D game concepts](#).

Field	Req	Type	Default	Meaning
version	opt	int	1	Data-format version.
spritesheet	req	path	—	Source spritesheet.
pivot	req	anchor name	—	Anchor used as the sprite's position pivot.
sequences	req	map name → sequence	—	Named playable sequences.

sequence : loop (bool, default false), frames ([{sprite, duration}]) where `sprite` is a frame id and `duration` is seconds.

Composite — `<name>.composite.yaml`

Worked example: [03 — 2D game concepts](#).

Field	Req	Type	Default	Meaning
version	opt	int	1	Data-format version.
root	req	{id, animation}	—	Root animated sprite (e.g. body).
children	opt	[child]	—	Attached animated sprites.

child: `id`, `parent (id)`, `animation (path)`, `parent_anchor` (anchor on the parent), `child_anchor` (anchor on the child, aligned to `parent_anchor`).

Dialog — `dialogs/<id>.lua`

Returns a node table. Worked example: [04 — Point & click concepts](#). The dialog id is both the file name and the speaking NPC's cast character id, so there is no speaker field in the file.

Field	Req	Type	Default	Meaning
<code>start</code>	req	node id	—	Entry node.
<code>on_enter</code>	opt	function	—	Setup callback.
<code>on_exit</code>	opt	function	—	Cleanup callback.
<i>(node id)</i>	—	node	—	Each remaining key is a node.

node: `npc` (string or `[string]`), and either `options` (`[option]`) or `to` (node id / `END`).

option (a list whose first element is the player line):

Field	Req	Type	Default	Meaning
<code>[1]</code>	req	string	—	Player line text.
<code>to</code>	req	node id <code>END</code>	—	Next node.
<code>when</code>	opt	function → bool	always	Visibility condition.
<code>run</code>	opt	function	—	Code run when selected.
<code>once</code>	opt	bool	<code>false</code>	Option disappears after use (persisted by <code>(node_id, option_index)</code>).
<code>silent</code>	opt	bool	<code>false</code>	Player line is not spoken aloud.

Inventory — `inventory.yaml`

Items are declared in `inventory.yaml` . Worked example: [04 — Point & click concepts](#).

Top-level field: `items` (req), a map of item id → item.

Field	Req	Type	Default	Meaning
name	req	string	—	Localized display name.
icon	opt	frame id path	name as text	Design-for image drawn in the SCUMM panel.
description	opt	string	—	Optional examine text.
affordances	req	[verb]	—	Verbs the UI may offer.
default_verb	opt	verb	look_at	Verb on a plain click; must be look_at or in affordances.
combinable	opt	bool	false	Whether use expects a second operand.

Inventory behavior — inventory.lua

Returns a table keyed by inventory item id. Each item table may define verb handlers using the same handler naming as room hotspots.

Handler kind	Example command	Signature
Single operand	Look at key	function()
Two operands: use	Use key with door	function(target_id)
Two operands: give	Give map to Stan	function(recipient_id)

For two-operand commands whose first operand is an inventory item, inventory handlers are checked before the second operand's hotspot handler.

Save file — GameState

The save payload and its YAML serialization are specified under "Make persistent state explicit" in [02 — Architecture overview](#); format, slots, and autosave policy are in [R8](#). Saves carry a save_version integer.